



Instituto Politécnico Nacional
Unidad Profesional Interdisciplinaria de Ingeniería
Campus Tlaxcala



Reporte Técnico

Edición musical condicionada por
instrucciones en lenguaje natural mediante
adaptación eficiente de modelos generativos
No. TT2026-1_IA

Presenta:

Homero Meneses Vázquez, Ingeniería en Inteligencia Artificial

Asesores:

Dr. Gerardo Taxis Taxis

Dr. Ricardo Ramos Aguilar

Tlaxcala, Tlaxcala, a **26 de junio de 2026**

"La Técnica al Servicio de la Patria"

AGRADECIMIENTOS

Agradezco a todos quienes han formado parte de este proyecto, a mis asesores por su apoyo y a mis seres queridos por su compañía.

RESUMEN

Este proyecto propone una actualización técnica del sistema de generación musical hacia un enfoque de edición de audio existente mediante instrucciones en lenguaje natural. En lugar de mantener la propuesta inicial basada en MusicLM, SeqGAN y búsqueda de Monte Carlo, el trabajo adopta una arquitectura reproducible compuesta por MusicGen-small, T5-base y EnCodec a 32 kHz, manteniendo a Transformers como familia arquitectónica principal.

La metodología se centra en adaptar el modelo mediante LoRA quirúrgico sobre las proyecciones Q , K y V de las capas de atención cruzada, junto con un módulo de fusión de audio con compuerta cero-inicializada. Este diseño permite inyectar el audio fuente como memoria sin modificar los pesos principales del modelo base, entrenando aproximadamente el 0.89 % de los parámetros totales. La evaluación se reorganiza alrededor de la edición correcta del audio fuente, empleando métricas como FAD, CLAP-score y tasa de cumplimiento de la instrucción, mientras que BLEU, MCC y otras métricas previas quedan como comparativa contextual.[1], [2]

Términos/Palabras Clave

- Modelos Generativos
- Transformers
- Aprendizaje Profundo
- LoRA
- Evaluación de música generada con IA

ÍNDICE GENERAL

1. Introducción	1
1.1. Antecedentes	1
1.2. Planteamiento del Problema	2
1.3. Objetivos	3
1.3.1. Objetivo General	3
1.3.2. Objetivos Específicos	3
1.4. Justificación	3
1.5. Hipótesis	4
1.6. Alcances	4
1.7. Limitaciones	4
2. Marco Teórico	5
2.1. Inteligencia Artificial Generativa	5
2.2. Redes Neuronales	5
2.3. Aprendizaje Profundo	5
2.4. Transformers	6
2.4.1. Modelos Basados en Transformers	6
2.5. MusicLM	7
2.5.1. Arquitectura General de MusicLM	7
2.6. BERT (Bidirectional Encoder Representations from Transformers) . . .	7
2.6.1. Proceso de Generación Musical	7
2.7. Tokens	7
2.7.1. Tokenización Acústica	8
2.8. Embeddings	8
2.8.1. Embeddings de Texto-Audio (Multimodales)	8
2.9. Embeddings Multimodales	8
2.10. RVQ	8
2.11. Fine-Tuning	8
2.11.1. Fine-Tuning Eficiente (PEFT)	9
2.11.2. Low-Rank Adaptation (LoRA)	9
2.12. Modelos Generativos Adversarios	9
2.13. Métricas Objetivas	10
2.13.1. FAD	10
2.13.2. CLAP Score	10
2.13.3. MuLan Cycle Consistency (MCC)	11

2.13.4. Bilingual Evaluation Understudy (BLEU)	12
2.13.5. Kullback–Leibler Divergence (KLD)	12
2.13.6. Kernel Audio Distance (KAD)	13
2.14. Evaluación Subjetiva	14
2.15. Evaluación Computacional	14
2.16. MIDI	14
2.17. Fundamentos de Música	14
2.17.1. Notación Musical	15
2.18. Parámetros	15
2.19. Abstracción Sonora	15
2.20. Estilo Musical	15
2.21. Género Musical	15
2.22. Música Instrumental	15
2.23. Corpus Multimodales	16
2.24. Bases de Datos Relacionales	16
2.24.1. Bases de Datos No Relacionales (NoSQL)	16
3. Estado del Arte	17
3.1. Bases de Datos	17
3.2. Representación discreta de audio y modelado autorregresivo	18
3.3. Modelos de difusión como alternativa no autorregresiva	18
3.4. Edición musical por instrucciones: paradigma <i>instruct</i>	18
3.5. Adaptación paramétrica eficiente y LoRA	19
3.6. Inyección de condicionamiento con compuerta cero-inicializada	19
3.7. Codificadores de texto para condicionamiento musical	19
3.8. Tabla comparativa del estado del arte	20
3.9. Evaluación cuantitativa del benchmark	20
3.9.1. Configuración experimental	20
3.9.2. Comparativa de modelos	21
3.9.3. Análisis estadístico del CLAP-score	21
3.9.4. Comparación visual mediante gráfica de radar	22
3.9.5. Discusión	23
3.10. Tendencias y desafíos abiertos	24
4. Metodología	25
4.1. Arquitectura general: flujo de edición musical	26
4.2. Componentes Transformer y configuraciones	26
4.2.1. Codificador de texto: T5-base	26
4.2.2. Tokenizador acústico: EnCodec 32 kHz	27
4.2.3. Decoder autorregresivo tipo MusicGen	27
4.2.4. Bloque Transformer implementado	27
4.3. Módulo de Fusión de Audio	28
4.4. Adaptación paramétrica: LoRA	29
4.5. Datos para la edición: modalidad, estandarización y ternas	31
4.5.1. Resolución de la inconsistencia de modalidad	31

4.5.2.	Registro canónico multimodal	31
4.5.3.	Datasets candidatos y trazabilidad de objetivos	32
4.5.4.	Generador del banco de ternas	32
4.5.5.	Integración de MoisesDB, pruebas y criterios de exclusión	33
4.5.6.	Captions ricos e instrucciones	33
4.6.	Ética, licenciamiento y uso responsable	34
4.7.	Régimen de entrenamiento	35
4.8.	Inferencia	35
4.9.	Protocolo de evaluación	36
4.10.	Componentes excluidos explícitamente	37
5.	Resultados	38
5.1.	Configuración experimental del prototipo	39
5.2.	Banco de edición construido y evaluación con anclas	41
5.2.1.	Composición del banco	41
5.2.2.	Evaluación de edición acotada por anclas	42
5.3.	Avance práctico	42
6.	Conclusiones	51
7.	Cronograma	52

CAPÍTULO 1

INTRODUCCIÓN

Un modelo de inteligencia artificial generativa es capaz de aprender las regularidades de un conjunto de datos para producir o transformar contenido. En audio musical, este proceso suele formularse como una tarea de modelado secuencial: la señal se codifica en representaciones discretas o latentes y un modelo aprende a predecir o modificar dichas representaciones de acuerdo con condiciones externas, como texto, melodías de referencia o instrucciones de edición.

La generación de audio por inteligencia artificial ha evolucionado desde modelos basados en espectrogramas hacia arquitecturas que tratan el sonido como una secuencia estructurada. SoundStream, EnCodec y variantes basadas en RVQGAN consolidaron el uso de cuantización residual para representar audio mediante tokens discretos, mientras que AudioLM, MusicLM y MusicGen demostraron que los Transformers pueden modelar dichas secuencias con coherencia temporal y semántica.[3], [4], [5], [6], [7], [8]

El presente trabajo reencuadra esa línea hacia edición de audio existente mediante instrucciones. En lugar de generar una pieza completa desde cero, el objetivo es modificar una pista fuente de acuerdo con una orden textual, preservando los elementos no solicitados. Esta formulación vuelve más relevante el condicionamiento multimodal, la adaptación paramétrica eficiente y la evaluación de cumplimiento de edición que la comparación general entre arquitecturas de generación pura.

1.1. Antecedentes

En el campo de la inteligencia artificial generativa aplicada al audio, hemos sido testigos de un avance secuencial significativo. Inicialmente, el progreso estuvo marcado por arquitecturas basadas en Transformers, para luego dar paso a una nueva generación de modelos basados en Difusión, que han elevado los estándares de calidad y fidelidad sonora. Sin duda, la era de los Transformers en la generación de música significa tratar el sonido como si fuera un lenguaje. En este grupo destacan MusicLM y MusicGen. MusicLM marcó un hito al utilizar un sistema jerárquico que modela la música en diferentes niveles de abstracción; su capacidad para traducir descripciones textuales complejas en piezas musicales coherentes de varios minutos mostró que los modelos generativos podían capturar no solo patrones rítmicos, sino también alineación semántica con la intención del usuario.

MusicGen simplificó esta línea mediante un Transformer de una sola etapa que opera sobre tokens de audio comprimidos con EnCodec. Esta formulación permite condicionar la generación con texto y, en ciertas variantes, con melodías de referencia, lo que la vuelve una base práctica para composición asistida y para extensiones posteriores hacia edición.[7], [8]

En paralelo, los modelos de difusión y flujo como Noise2Music, Moûsai, AudioLDM, AudioLDM2, MusicLDM, Make-An-Audio, TangoFlux y ERNIE-Music elevaron la fidelidad acústica mediante procesos de denoising, mezcla latente, optimización por preferencias o generación en espacios latentes.[9], [10], [11], [12], [13], [14], [15], [16] Sin embargo, su uso para edición puntual de audio requiere mecanismos adicionales, como inversión, máscaras o condicionamientos auxiliares.

La tendencia más cercana a este proyecto es la edición musical por instrucciones. En este paradigma, el usuario entrega una pista fuente y una orden textual; el sistema debe producir una versión modificada que cumpla la operación solicitada. Trabajos como AUDIT, InstructME e Instruct-MusicGen demuestran que esta tarea puede abordarse con supervisión directa y adaptación eficiente, sin depender de un esquema adversarial.[17], [18], [19]

Por esta razón, el documento adopta MusicGen-small como modelo base, T5-base como codificador de instrucciones, EnCodec como tokenizador acústico y LoRA como mecanismo de ajuste eficiente. Esta combinación permite mantener el núcleo Transformer del modelo, reducir el número de parámetros entrenables y orientar la evaluación hacia tres criterios: calidad acústica, alineación texto-audio y cumplimiento de la edición.

1.2. Planteamiento del Problema

La edición musical por instrucciones enfrenta cuatro retos principales:

1. **Condicionamiento multimodal:** el sistema debe interpretar simultáneamente una pista de audio fuente y una instrucción textual, preservando los elementos no solicitados por el usuario.
2. **Reproducibilidad:** modelos como MusicLM no liberan pesos completos, por lo que resulta necesario trabajar con arquitecturas disponibles públicamente, como MusicGen y EnCodec.
3. **Eficiencia computacional:** ajustar modelos de cientos de millones de parámetros exige estrategias de adaptación ligera que reduzcan memoria, tiempo de entrenamiento y costo de hardware.
4. **Evaluación diagnóstica:** la calidad musical no puede evaluarse únicamente con métricas de generación pura; la tarea requiere medir calidad acústica, alineación texto-audio y cumplimiento efectivo de la edición solicitada.

1.3. Objetivos

1.3.1. Objetivo General

Diseñar y analizar un sistema de edición musical por instrucciones en lenguaje natural basado en MusicGen-small, T5-base, EnCodec y adaptación LoRA, evaluando su calidad acústica, alineación semántica y capacidad para aplicar correctamente operaciones de edición sobre audio existente.

1.3.2. Objetivos Específicos

1. Construir un banco multimodal de ternas compuesto por audio fuente, instrucción de edición y audio objetivo, implementando WAV 32 kHz mono, *stems*, atributos acústicos e instrucciones; MIDI transcrito y tokens EnCodec.
2. Integrar un módulo de fusión de audio por atención cruzada con compuerta cero-inicializada para inyectar el audio fuente sin degradar el modelo base.
3. Aplicar LoRA quirúrgico únicamente en las proyecciones Q , K y V de las atenciones cruzadas para entrenar menos del 1 % de los parámetros totales.
4. Preparar ternas de entrenamiento compuestas por audio original, instrucción de edición y audio objetivo, codificadas con T5 y EnCodec.
5. Evaluar el sistema mediante FAD, CLAP-score y tasa de cumplimiento de la instrucción, manteniendo métricas heredadas solo como comparación contextual.

1.4. Justificación

De acuerdo con la federación internacional de la industria fonográfica (IFPI), más de 2,500 millones de personas en el mundo practican la música de alguna manera. El Global Music Report 2025 destaca un aumento del 4.8 % en los ingresos globales en 2024. El streaming representa el 69 % de los ingresos globales de la industria, y en latinoamérica, el crecimiento de ingresos por música grabada ha sido del 22 %, con México en el puesto número 10 de los principales mercados musicales a nivel global.[20]

“La inteligencia artificial puede acercar aún más a los fans a sus artistas favoritos, permitiéndoles participar en la música como nunca antes habían podido. Y los grandes artistas se involucrarán para crear arte que cambie la cultura, lo que generará entusiasmo y hará crecer el negocio.” Dennis Kooker, Presidente de Negocios Digitales Globales y Ventas en EE. UU. de Sony Music.[20]

Hoy en día, existe un amplio abanico de herramientas para automatizar la creación musical. Desde programas para crear partituras hasta estaciones de trabajo de audio digital (DAW), la tecnología ha democratizado la creación musical. De acuerdo con el nivel de trabajo del creador, existen plataformas que generan canciones completas con un solo *prompt*, así como sistemas instrumentales y modelos abiertos que pueden adaptarse a tareas particulares.[21]

La edición musical por instrucciones representa un avance significativo dentro de la inteligencia artificial aplicada a la creatividad, porque permite modificar material sonoro existente sin reconstruir una pieza desde cero. Esta orientación es más cercana a flujos reales de producción musical, donde el usuario suele necesitar operaciones concretas como eliminar un instrumento, conservar una línea de bajo, modificar el estilo o ajustar una sección específica.

La adopción de MusicGen-small, T5-base, EnCodec y LoRA contribuye al campo de la *IA generativa* al ofrecer un marco metodológico replicable, eficiente y viable para entornos académicos. Además, la evaluación mediante FAD, CLAP-score y cumplimiento de instrucción permite analizar no solo la calidad acústica, sino también si el sistema realiza efectivamente la edición solicitada.

Además, la investigación fortalece el desarrollo de herramientas para músicos e investigadores al facilitar procesos de edición asistida, reutilización de material sonoro y experimentación con modelos generativos sin requerir el re-entrenamiento completo de arquitecturas de gran escala.

1.5. Hipótesis

Es posible habilitar la edición controlada de audio entrenando menos del 1 % de los parámetros del sistema mediante adaptadores de bajo rango (LoRA quirúrgico) y fusión por atención cruzada, sin degradar la calidad del modelo base y con un costo computacional viable para un presupuesto académico.

1.6. Alcances

El proyecto se enfoca en la edición de audio musical existente mediante instrucciones textuales. Su alcance incluye el análisis de MusicGen-small como modelo base, el uso de T5-base para representar instrucciones, la codificación acústica con EnCodec a 32 kHz, la incorporación de un módulo de fusión de audio por atención cruzada y la adaptación eficiente mediante LoRA quirúrgico. La evaluación se concentra en calidad acústica, alineación texto-audio y corrección de la edición.[1], [2]

1.7. Limitaciones

El sistema no entrena MusicLM ni utiliza SeqGAN, discriminadores adversariales o búsqueda de Monte Carlo. Tampoco aborda una evaluación multicultural con expertos como componente central del protocolo, sino que la deja como trabajo futuro, especialmente por la relevancia de estudiar sesgos representacionales y adaptabilidad transcultural en modelos de generación musical.[22] La calidad final dependerá de la disponibilidad de pares supervisados de edición (audio original, instrucción y audio objetivo), de la cobertura de las instrucciones disponibles y de la capacidad de los clasificadores auxiliares para verificar operaciones como presencia o ausencia de instrumentos.

CAPÍTULO 2

MARCO TEÓRICO

2.1. Inteligencia Artificial Generativa

Los modelos generativos para audio tienen como objetivo sintetizar señales sonoras que reproduzcan las propiedades temporales y espectrales del audio real. A diferencia de otros dominios como la imagen, el audio presenta una naturaleza altamente secuencial y dependiente del tiempo, lo que implica retos adicionales en la modelación de dependencias a largo plazo, dinámica temporal y coherencia armónica.

Estos modelos pueden operar directamente sobre la forma de onda, sobre representaciones intermedias como espectrogramas, o mediante representaciones simbólicas como MIDI o tokens discretos de audio. La elección de la representación impacta directamente en la complejidad computacional y en la calidad perceptual del audio generado.

2.2. Redes Neuronales

Sistemas computacionales inspirados en la estructura del cerebro humano, compuestos por capas de nodos interconectados (neuronas artificiales) que procesan información mediante funciones de activación y pesos ajustables. Constituyen la base de los modelos de aprendizaje profundo modernos.

2.3. Aprendizaje Profundo

El aprendizaje profundo es un subcampo del aprendizaje automático basado en redes neuronales artificiales con múltiples capas ocultas. Estas arquitecturas permiten aprender representaciones jerárquicas de los datos, capturando patrones de bajo y alto nivel de forma automática.

En el contexto del audio, el aprendizaje profundo ha permitido modelar relaciones complejas entre características acústicas, como frecuencia, amplitud y timbre, facilitando tareas como síntesis de voz, generación musical y transformación de audio. Redes recurrentes, convolucionales y, más recientemente, Transformers, han demostrado ser especialmente eficaces para capturar la naturaleza secuencial del audio.

2.4. Transformers

Arquitectura de red neuronal basada en mecanismos de atención que permite procesar secuencias completas de datos en paralelo, eliminando la necesidad de procesamiento secuencial de modelos anteriores como RNN. Su capacidad para capturar dependencias a largo plazo los hace ideales para tareas de generación de lenguaje y audio.

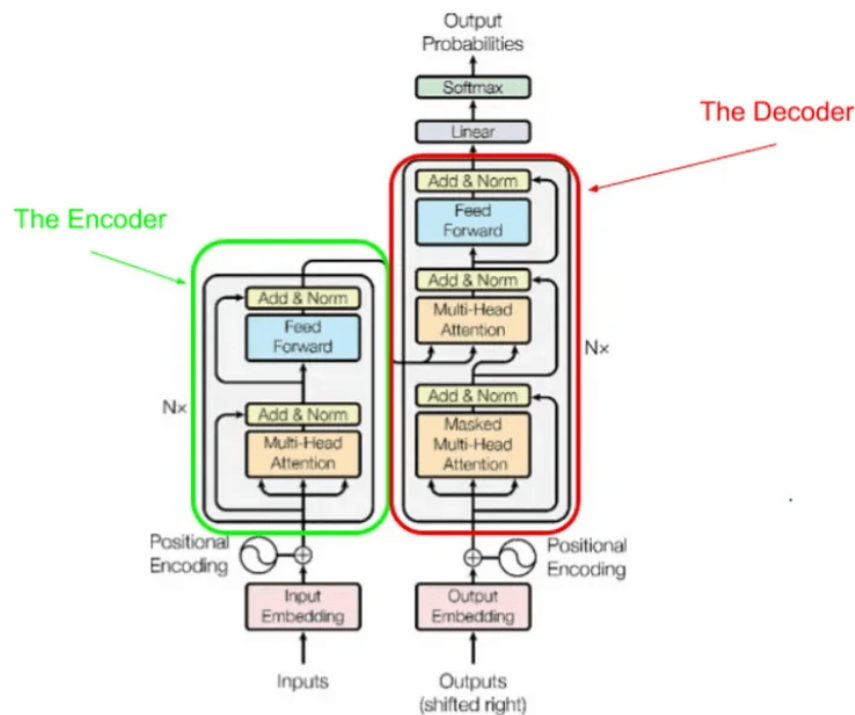


Figura 2.1: Transformers en IA (Fuente: [23])

2.4.1. Modelos Basados en Transformers

Los Transformers son una arquitectura de redes neuronales introducida para modelar secuencias mediante mecanismos de autoatención. A diferencia de las redes recurrentes, los Transformers permiten procesar secuencias completas en paralelo, facilitando el aprendizaje de dependencias a largo plazo. En generación de audio, los Transformers han sido adaptados para modelar secuencias discretas de tokens acústicos, permitiendo una mayor escalabilidad y calidad en la generación. Su capacidad para capturar contexto global resulta fundamental en tareas musicales, donde la coherencia melódica y estructural es esencial.

2.5. MusicLM

MusicLM es un modelo generativo de música basado en arquitecturas Transformer, diseñado para generar música de alta fidelidad a partir de descripciones textuales. Este modelo representa un avance significativo al integrar procesamiento de lenguaje natural y generación de audio, permitiendo una interacción semántica directa entre el usuario y el sistema generativo. El enfoque de MusicLM se basa en la generación jerárquica de audio mediante representaciones discretas, lo que permite manejar secuencias largas manteniendo coherencia musical.

2.5.1. Arquitectura General de MusicLM

La arquitectura de MusicLM se compone de múltiples modelos Transformer organizados jerárquicamente. En una primera etapa, el texto de entrada es procesado para generar una representación semántica. Posteriormente, esta representación guía la generación de tokens de audio en distintos niveles de resolución temporal. Este enfoque jerárquico permite separar la estructura musical global de los detalles acústicos finos, reduciendo la complejidad del proceso de generación y mejorando la calidad del resultado final.

2.6. BERT (Bidirectional Encoder Representations from Transformers)

Modelo transformer bidireccional pre-entrenado que aprende representaciones contextuales mediante el enmascaramiento de tokens. En el contexto de audio, variantes como w2v-BERT (wav2vec-BERT) combinan aprendizaje auto-supervisado de representaciones acústicas con la arquitectura BERT.

2.6.1. Proceso de Generación Musical

El proceso de generación musical en MusicLM es autoregresivo y condicionado por texto. A partir de la descripción textual, el modelo genera progresivamente tokens de audio que posteriormente son decodificados para producir la señal sonora final. Este proceso permite generar música coherente con el contexto semántico proporcionado, manteniendo consistencia rítmica, armónica y estilística.

2.7. Tokens

Unidades discretas de información que representan fragmentos de datos continuos. En procesamiento de audio, los tokens pueden codificar características acústicas, semánticas o simbólicas de la señal sonora, permitiendo su manipulación mediante modelos de lenguaje.

2.7.1. Tokenización Acústica

Proceso de conversión de señales de audio continuas en secuencias discretas de tokens mediante técnicas como cuantización vectorial (VQ-VAE) o codificación neuronal. Esta representación discreta facilita la aplicación de arquitecturas transformer originalmente diseñadas para texto.

2.8. Embeddings

Representaciones vectoriales densas de datos en espacios de dimensionalidad reducida que capturan propiedades semánticas y estructurales. Los embeddings permiten que elementos similares tengan representaciones cercanas en el espacio vectorial.

2.8.1. Embeddings de Texto-Audio (Multimodales)

Representaciones vectoriales compartidas que proyectan tanto texto como audio a un espacio común, permitiendo establecer correspondencias semánticas entre modalidades. Ejemplos incluyen MuLan y CLAP, que facilitan el condicionamiento textual en generación de audio.

2.9. Embeddings Multimodales

MuLan proyecta música y texto a un espacio vectorial compartido, permitiendo condicionamiento textual robusto sin pares exactos texto-audio.

2.10. RVQ

La compresión de datos es crucial para construir un modelo eficiente computacionalmente hablando. Por ello La cuantificación de vectores residuales (RVQ) se centra en descomponer datos complejos en partes más simples, divide la información en múltiples vectores numéricos corrigiendo el error residual en cada paso. [24]

2.11. Fine-Tuning

El fine-tuning tradicional consiste en reentrenar parcialmente o completamente un modelo preentrenado utilizando un nuevo conjunto de datos. Aunque esta técnica permite una adaptación efectiva, suele ser costosa en términos de memoria, tiempo y recursos computacionales, especialmente en modelos de gran escala.

2.11.1. Fine-Tuning Eficiente (PEFT)

Las técnicas LoRA y AdapterFusion permiten ajustar modelos grandes mediante capas adicionales de baja dimensionalidad, reduciendo costos de entrenamiento y almacenamiento.

2.11.2. Low-Rank Adaptation (LoRA)

LoRA es una técnica de adaptación eficiente que introduce matrices de bajo rango en capas específicas del modelo, manteniendo congelados los pesos originales. Esto permite reducir significativamente el número de parámetros entrenables, disminuyendo el costo computacional sin sacrificar de forma notable el rendimiento.

2.12. Modelos Generativos Adversarios

Los modelos generativos adversarios (GANs) son una clase de modelos compuestos por dos redes neuronales: un generador y un discriminador, que compiten en un proceso de entrenamiento adversarial. Este enfoque ha demostrado ser eficaz para generar datos realistas en diversos dominios. El generador aprende a producir muestras sintéticas mientras que el discriminador evalúa si una muestra es real o generada. El entrenamiento se basa en un juego de suma cero, donde ambos modelos mejoran progresivamente.

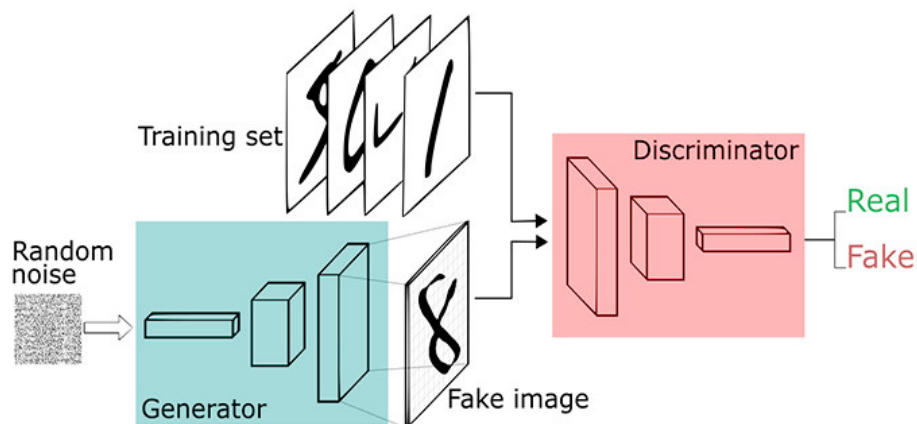


Figura 2.2: Ejemplo de arquitectura de GAN (Fuente: [25])

2.13. Métricas Objetivas

Las métricas objetivas se basan en comparaciones matemáticas entre señales, como distancia espectral, entropía o similitud de características acústicas.

2.13.1. FAD

La Fréchet Audio Distance, es un instrumento matemático que mide la similitud entre una distribución de audios artificiales y una de audios de referencia (reales), comparando sus medias y covarianzas. En IA generativa y herramientas de mejora de audio, un puntaje FAD más bajo significa que el audio generado es más realista y de mayor calidad.

Proceso de Funcionamiento de FAD (Fréchet Audio Distance)

Incrustaciones (Embeddings): Transforma las ondas de sonido en representaciones numéricas de alta dimensión utilizando modelos como VGGish o PANN.

Distribución estadística: Calcula y modela estas representaciones como una distribución gaussiana, definida por su media y su matriz de covarianza.

Basada en *embeddings* extraídos por una red neuronal preentrenada (generalmente VGGish). Modela ambas distribuciones como gaussianas.

Sean:

- $\mathcal{X}_r$: El conjunto de audios reales.
- $\mathcal{X}_g$: El conjunto de audios generados.
- μ_r, Σ_r : La media y la matriz de covarianza de los *embeddings* reales.
- μ_g, Σ_g : La media y la matriz de covarianza de los *embeddings* generados.

La FAD se define mediante la distancia de Fréchet entre las dos distribuciones:

$$\text{FAD}(\mathcal{X}_r, \mathcal{X}_g) = \|\mu_r - \mu_g\|_2^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}) \quad (2.1)$$

[1]

2.13.2. CLAP Score

El *CLAP Score* se deriva del modelo de preentrenamiento contrastivo audio-lenguaje (*Contrastive Language-Audio Pretraining*). Su arquitectura proyecta señales de audio y textos descriptivos a un espacio vectorial compartido mediante aprendizaje contrastivo, lo que permite comparar directamente ambas modalidades.[2] En este contexto, el *CLAP Score* mide la similitud semántica entre el audio generado o separado y una consulta textual mediante la similitud coseno entre sus *embeddings* en dicho espacio compartido. A diferencia de métricas tradicionales basadas en relación señal-distorsión, como SDR, esta métrica no requiere un audio de referencia, por lo que resulta especialmente útil en escenarios *reference-free* donde no existe un audio canónico contra el cual comparar.

Variantes del modelo En la literatura se reportan múltiples variantes del modelo CLAP. Entre ellas, CLAP LAION Music (CLAP-M), entrenado específicamente con contenido musical, ha mostrado mejor desempeño que el CLAP estándar y que CLAP LAION Audio (CLAP-A) en tareas de evaluación de música generativa, particularmente cuando se utiliza para apoyar el cálculo de métricas perceptuales como FAD. Estas diferencias suelen atribuirse al tamaño, la diversidad y la especialización musical de los datos de entrenamiento, así como al objetivo contrastivo que favorece la captura de atributos perceptualmente relevantes.

Proceso de cómputo El cálculo del *CLAP Score* comprende cuatro etapas principales:

- Codificación del audio generado en un vector de *embedding* mediante el codificador de audio del modelo CLAP.
- Codificación del texto descriptivo o *prompt* en un vector de *embedding* mediante el codificador de texto.
- Cálculo de la similitud coseno entre ambos vectores en el espacio compartido.
- Promediado del puntaje sobre el conjunto de evaluación.

Definición matemática Sean $f_A(a_g) \in \mathbb{R}^d$ el *embedding* del audio generado y $f_T(t) \in \mathbb{R}^d$ el *embedding* del texto asociado. Entonces, el puntaje se define como:

$$\text{CLAPScore}(a_g, t) = \frac{f_A(a_g) \cdot f_T(t)}{\|f_A(a_g)\| \|f_T(t)\|} \quad (2.2)$$

Un valor cercano a 1.0 indica alta coherencia semántica entre el audio y el texto que lo describe, mientras que valores próximos a 0 reflejan baja correspondencia. Debido a que su implementación de referencia se distribuye en código abierto, esta métrica también resulta conveniente para protocolos reproducibles de evaluación automática.

2.13.3. MuLan Cycle Consistency (MCC)

La MuLan Cycle Consistency (MCC) evalúa la coherencia semántica entre un texto t y el audio generado a_g , utilizando un espacio de *embeddings* multimodales compartido aprendido por el modelo MuLan. Esta métrica es fundamental para determinar qué tan bien el modelo "entiende" las instrucciones textuales.

Definición matemática Sean:

- $f_T(t) \in \mathbb{R}^d$: El *embedding* del texto de entrada.
- $f_A(a_g) \in \mathbb{R}^d$: El *embedding* del audio generado.

La similitud coseno entre ambas representaciones vectoriales:

$$\text{MCC}(t, a_g) = \frac{f_T(t) \cdot f_A(a_g)}{\|f_T(t)\| \|f_A(a_g)\|} \quad (2.3)$$

Donde valores cercanos a 1 indican una alta fidelidad semántica texto-audio, validando que el contenido musical generado se alinea con la descripción proporcionada por el usuario. [26]

2.13.4. Bilingual Evaluation Understudy (BLEU)

La métrica BLEU es un método de evaluación basado en referencia que, aunque fue diseñado originalmente para la traducción automática, se utiliza en la generación musical simbólica para medir la coincidencia de n -gramas entre las secuencias generadas y las de referencia.

Definición matemática Para un conjunto de n -gramas determinado, la puntuación BLEU se calcula como:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (2.4)$$

Donde los parámetros se definen de la siguiente manera:

- p_n : Es la precisión modificada de los n -gramas.
- w_n : Son los pesos de importancia, usualmente definidos como $\frac{1}{N}$.
- BP: Es el factor de penalización por brevedad (*Brevity Penalty*), que evita puntuaciones infladas para secuencias excesivamente cortas.

El factor BP se calcula mediante la siguiente función por partes:

$$\text{BP} = \begin{cases} 1 & \text{si } c > r \\ e^{(1-r/c)} & \text{si } c \leq r \end{cases} \quad (2.5)$$

Aquí, c representa la longitud de la secuencia generada (candidata) y r es la longitud de la secuencia de referencia. [27]

2.13.5. Kullback–Leibler Divergence (KLD)

La Divergencia de Kullback–Leibler (KLD), también denominada entropía relativa, es una medida de teoría de la información que cuantifica cuánto difiere una distribución de probabilidad Q con respecto a una distribución de referencia P . La KLD también se relaciona con familias más generales de divergencias, como la divergencia de Rényi, usadas para caracterizar diferencias entre distribuciones de probabilidad.[28] En evaluación de audio generativo, esta métrica permite comparar qué tan distinta es la distribución acústica del audio generado frente a la del audio real.

Fundamento matemático Para distribuciones discretas, la KLD se define como:

$$D_{KL}(P\|Q) = \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{P(x)}{Q(x)} \right) \quad (2.6)$$

Donde:

- $P(x)$ representa la distribución del audio real o de referencia.
- $Q(x)$ representa la distribución inducida por el audio generado.
- $\mathcal{X}$ es el espacio de eventos o clases acústicas consideradas.

La KLD es una medida asimétrica, es decir, en general $D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$, y solo toma el valor cero cuando ambas distribuciones son idénticas. Valores positivos mayores indican que el modelo generativo produce una distribución de clases acústicas distinta a la del conjunto de referencia.

Aplicación en evaluación de audio En la práctica, la KLD no suele calcularse directamente sobre la forma de onda, sino sobre distribuciones de probabilidad de clases acústicas obtenidas mediante un clasificador preentrenado. Bajo este enfoque, cada audio se transforma primero en una distribución sobre etiquetas; posteriormente, se compara la distribución estimada para el audio generado con la distribución del audio de referencia y se promedia la divergencia sobre todo el conjunto de evaluación. Una KLD baja indica que el audio generado conserva propiedades acústicas similares a las del corpus de referencia según las categorías del clasificador, mientras que una KLD alta sugiere un alejamiento estadístico respecto a dicho corpus.

Interpretación Es importante distinguir la orientación del cálculo. La forma $D_{KL}(P\|Q)$ penaliza con mayor severidad los casos en que la distribución generada Q asigna probabilidad nula o muy baja a regiones donde P sí concentra masa. En evaluación generativa, esta propiedad es útil para detectar modos del conjunto real que el modelo no logra cubrir. En consecuencia, valores cercanos a cero indican que el modelo ha capturado adecuadamente la estructura estadística del dominio objetivo, mientras que valores altos señalan deficiencias en cobertura o fidelidad acústica.[29]

2.13.6. Kernel Audio Distance (KAD)

La Kernel Audio Distance (KAD) es una métrica de evaluación que, a diferencia de la FAD, no asume que las distribuciones de los *embeddings* siguen una forma gaussiana. KAD utiliza la Discrepancia Máxima de Medias (MMD) para comparar las distribuciones de audio en un espacio de características inducido por un *kernel* positivo definido.

Definición matemática Sea $\phi(x)$ un mapeo implícito a un espacio de Hilbert (RKHS) inducido por un *kernel* $k(x, x')$. La distancia KAD al cuadrado entre la distribución real P y la generada Q se define como:

$$\text{KAD}^2(P, Q) = \mathbb{E}_{x, x' \sim P}[k(x, x')] + \mathbb{E}_{y, y' \sim Q}[k(y, y')] - 2\mathbb{E}_{x \sim P, y \sim Q}[k(x, y)] \quad (2.7)$$

Donde:

- $\mathbb{E}$: Representa el valor esperado sobre las muestras de las distribuciones.
- $k(x, y)$: Es la función de *kernel* (comúnmente un *kernel* polinomial o RBF).
- x, x' : Son muestras de audio real.
- y, y' : Son muestras de audio generado.

La principal ventaja o aporte de KAD es su capacidad para capturar momentos de orden superior en los datos, proporcionando una medición más representativa cuando las distribuciones presentan sesgos o colas pesadas. [30]

2.14. Evaluación Subjetiva

La evaluación subjetiva involucra la percepción humana, considerando aspectos como la calidad, la coherencia musical y el estilo musical.

2.15. Evaluación Computacional

La evaluación computacional analiza el consumo de recursos, el tiempo de entrenamiento, la memoria utilizada y la eficiencia del modelo durante la generación.

2.16. MIDI

MIDI es un estándar de representación musical simbólica que describe eventos musicales como notas, duración e intensidad, sin almacenar audio directamente. Su uso es común en sistemas de generación musical debido a su bajo costo computacional y facilidad de manipulación.

2.17. Fundamentos de Música

Los fundamentos de música proporcionan el marco teórico necesario para analizar y generar estructuras musicales coherentes, incluyendo ritmo, melodía, armonía y forma.

2.17.1. Notación Musical

La notación musical es un sistema simbólico que permite representar gráficamente sonidos musicales, indicando altura, duración, intensidad y estructura temporal. En el contexto de generación automática, la notación musical facilita la evaluación estructural y estilística de la música generada.

2.18. Parámetros

Variables numéricas aprendibles que definen el comportamiento de una red neuronal. En modelos de generación musical, el número de parámetros (millones o miles de millones) determina la capacidad del modelo para capturar patrones complejos, aunque también impacta los requisitos computacionales.

2.19. Abstracción Sonora

Representación de características de audio en distintos niveles de granularidad, desde atributos acústicos de bajo nivel (espectrogramas, envolventes) hasta conceptos semánticos de alto nivel (género, emoción, estructura musical). Los modelos jerárquicos explotan esta organización multinivel.

2.20. Estilo Musical

Conjunto de características distintivas que definen la forma de expresión musical, incluyendo patrones rítmicos, armónicos, tímbricos y estructurales propios de un artista, época o movimiento cultural específico.

2.21. Género Musical

Categoría clasificatoria de música basada en convenciones compartidas de forma, instrumentación, estructura y contexto cultural. Ejemplos incluyen rock, jazz, clásica, electrónica, entre otros.

2.22. Música Instrumental

Composición musical que carece de componente vocal, enfocándose exclusivamente en la expresión mediante instrumentos musicales. Presenta desafíos específicos para modelos generativos al requerir coherencia estructural sin apoyo lírico.

2.23. Corpus Multimodales

Conjuntos de datos que integran múltiples modalidades de información (audio, texto, metadatos, partituras) con anotaciones y correspondencias entre ellas. Fundamentales para entrenar modelos que relacionan descripciones textuales con contenido sonoro.

2.24. Bases de Datos Relacionales

Sistemas de gestión de datos estructurados mediante tablas con relaciones definidas entre ellas, utilizando SQL para consultas. Útiles para almacenar metadatos estructurados de corpus musicales (artista, álbum, año, género).

2.24.1. Bases de Datos No Relacionales (NoSQL)

Sistemas de almacenamiento flexibles que no requieren esquemas fijos, incluyendo bases documentales (MongoDB), de grafos o clave-valor. Apropriadas para datos heterogéneos como embeddings, anotaciones variables o estructuras jerárquicas complejas.

■ Lenguajes y Herramientas

- *Python*: Lenguaje principal para machine learning, con ecosistema rico en bibliotecas (PyTorch, TensorFlow, librosa, transformers).
- *PyTorch/TensorFlow*: Frameworks de aprendizaje profundo para construcción, entrenamiento y despliegue de modelos neuronales.
- *Hugging Face Transformers*: Biblioteca que proporciona implementaciones pre-entrenadas de modelos transformer y facilita fine-tuning.
- *librosa*: Biblioteca Python para análisis y procesamiento de señales de audio.
- *Docker*: Plataforma de contenedorización para garantizar reproducibilidad de entornos de desarrollo y despliegue.

CAPÍTULO 3

ESTADO DEL ARTE

La generación automática de contenido musical ha experimentado una transformación profunda en la última década. Los primeros enfoques, basados en reglas explícitas, gramáticas formales y modelos probabilísticos simples, han sido progresivamente desplazados por arquitecturas de aprendizaje profundo capaces de capturar patrones complejos en grandes volúmenes de datos. Uno de los cambios conceptuales más relevantes ha sido el abandono de la música como mera señal continua para su reinterpretación como una secuencia de representaciones discretas. Este nuevo paradigma ha permitido transferir con éxito los avances logrados por los Grandes Modelos de Lenguaje (LLMs) al dominio del audio, habilitando la generación de estructuras musicales coherentes a largo plazo, algo que resultaba inalcanzable para los enfoques tradicionales de síntesis sonora.

3.1. Bases de Datos

La robustez de los modelos generativos depende directamente de la escala, licencia, calidad y modalidad de los conjuntos de datos utilizados. Para música generativa se distinguen tres familias principales: corpus masivos de audio para preentrenamiento acústico y semántico, pares música–texto para alineación entre lenguaje natural y audio, y colecciones simbólicas o MIDI para estructura musical de alto nivel. AudioSet es útil por su diversidad acústica; MusicCaps se emplea con frecuencia como referencia de evaluación texto–música, aunque no constituye por sí solo un estándar universal; y Lakh MIDI o Slakh2100 aportan información simbólica, mezclas sintéticas y/o *stems* útiles para estudiar estructura, tiempo e instrumentación.[7], [31], [32], [33]

En este trabajo la selección de datos no se reduce a una sola modalidad. La edición por instrucciones requiere conservar atributos acústicos como timbre, mezcla y presencia de instrumentos, por lo que el banco experimental se formaliza como un registro multimodal descrito en la Sección 4.5. Esta decisión evita que MIDI sea tratado como representación única y permite que cada operación de edición elija la modalidad apropiada para construir el audio objetivo.

3.2. Representación discreta de audio y modelado autorregresivo

Una estrategia ampliamente adoptada para tratar audio como secuencia tokenizada se apoya en cuantizadores residuales como SoundStream, EnCodec y RVQGAN, que convierten una forma de onda en una rejilla de tokens discretos organizada en varios *codebooks* paralelos.[3], [4], [5] Sobre esa representación, AudioLM y MusicLM propusieron jerarquías de tokens semánticos y acústicos modeladas por Transformers autorregresivos.[6], [7] MusicGen simplificó esta línea al usar un único Transformer entrenado sobre *codes* de EnCodec a 32 kHz con cuatro *codebooks* y un patrón de retraso entre ellos; esta disponibilidad pública y su tamaño manejable justifican su uso como modelo base en el presente trabajo.[8]

3.3. Modelos de difusión como alternativa no autorregresiva

En paralelo, modelos de difusión y flujo como Noise2Music, Moûsai, AudioLDM, AudioLDM2, MusicLDM, Make-An-Audio y TangoFlux generan audio mediante refinamiento iterativo de ruido, estrategias latentes, mezcla sincronizada al pulso o flujo generativo.[9], [10], [11], [12], [13], [14], [15] Aunque suelen producir texturas naturales y fidelidad competitiva, su inferencia requiere múltiples pasos de denoising y la edición puntual de una pista preexistente no es nativa. En este documento se consideran como contexto comparativo, no como arquitectura base, porque la edición guiada por instrucciones se modela de forma más directa con el paradigma autorregresivo de MusicGen.

3.4. Edición musical por instrucciones: paradigma *instruct*

El giro reciente más relevante para este proyecto es el paradigma *instruct*, en el que el modelo recibe simultáneamente un audio fuente y una orden textual, y debe producir el audio editado. AUDIT formuló esta idea para edición de audio general mediante difusión latente; InstructME la trasladó al dominio musical con operaciones entre *stems*; e Instruct-MusicGen la implementó sobre MusicGen mediante un módulo de fusión de audio por atención cruzada y adaptadores LoRA en capas de atención.[17], [18], [19] Este último trabajo constituye la referencia más cercana para el sistema propuesto.

3.5. Adaptación paramétrica eficiente y LoRA

Low-Rank Adaptation (LoRA) reemplaza la actualización completa de una matriz W por una corrección de bajo rango, de modo que el cómputo se expresa como:

$$y = Wx + \frac{\alpha}{r}B(Ax), \quad (3.1)$$

donde W permanece congelada y únicamente se entrenan las matrices A y B . [34] Al inicializar B en cero, el delta inicial es nulo y se preserva el comportamiento del modelo pre-entrenado. En MusicGen, las proyecciones Q , K y V de la atención cruzada concentran el puente entre el condicionamiento textual o acústico y los tokens musicales, por lo que este trabajo adopta una estrategia de LoRA quirúrgico con rango $r = 16$ y factor $\alpha = 32$, aplicada solo a las dos atenciones cruzadas del decoder.

3.6. Inyección de condicionamiento con compuerta cero-inicializada

Al añadir un módulo de fusión de audio sobre un modelo pre-entrenado existe el riesgo de degradar la síntesis por introducir señales aleatorias al inicio del entrenamiento. Para evitarlo, se adopta una compuerta cero-inicializada inspirada en ControlNet y Flamingo: la salida del módulo se multiplica por $\tanh(g)$, con $g = 0$ al arranque. [35], [36] Así, el primer *forward pass* reproduce el comportamiento de MusicGen base, y la apertura de la compuerta se aprende únicamente si el audio fuente reduce la pérdida sobre el objetivo.

3.7. Codificadores de texto para condicionamiento musical

MusicLM y MusicGen utilizan codificadores de texto basados en Transformers pre-entrenados a gran escala; MusicGen emplea T5. [8], [37] En este trabajo no se aplica *mean-pooling* a la salida de T5, sino que se conserva la secuencia completa de estados ocultos ($L, 768$). Esta decisión es necesaria en instrucciones como *remove the drums but keep the bassline*, donde el verbo, el objeto y la cláusula de preservación deben mantener identidades separadas para que la atención cruzada pueda asociarlos con regiones distintas de los tokens musicales.

3.8. Tabla comparativa del estado del arte

Modelo	Familia	Tarea	Base	Entrenable
MusicLM	Transformer jerárquico	Generación T→A	AudioLM + MuLan	Todo
MusicGen	Transformer 1 etapa	Generación T→A	EnCodec 32 kHz	Todo
AudioLDM2	Difusión latente	Generación T→A	VAE + UNet	Todo
AUDIT	Difusión latente	Edición A+T→A	AudioLDM	Todo
InstructME	Difusión latente	Edición A+T→A	AudioLDM	Todo
Instruct-MusicGen	Transformer + Lo-RA	Edición A+T→A	MusicGen + T5	≈ 8 %
Este trabajo	Transformer + Lo-RA	Edición A+T→A	MusicGen-small + T5-base + EnCodec-32k	0.89 %

Cuadro 3.1: Posicionamiento del presente trabajo frente a sistemas representativos. T representa texto y A representa audio.

3.9. Evaluación cuantitativa del benchmark

Antes de adaptar un modelo a la tarea de edición por instrucciones, se realizó un estudio comparativo para justificar empíricamente la elección de la familia de modelo base. El objetivo de esta sección no es evaluar la edición, sino contrastar la capacidad de *generación* texto–audio de varios candidatos y verificar que la familia MusicGen ofrece la mejor base sobre la cual construir el sistema de edición. Aunque MusicGen-medium obtiene el mejor desempeño absoluto, este trabajo adopta MusicGen-small como base por restricciones de presupuesto de cómputo; como se muestra a continuación, MusicGen-small aún supera a los dos baselines evaluados.

3.9.1. Configuración experimental

Se evaluaron cuatro modelos de generación musical sobre un banco de 100 prompts extraídos de MusicCaps [7], distribuidos uniformemente en 10 géneros: *ambient*, *classical*, *electronic*, *hip-hop*, *jazz*, *latin*, *pop*, *reggaeton*, *rock* y *world* (10 prompts por género). Los modelos comparados fueron MusicGen-medium, MusicGen-small, AudioLDM2 [12] y Stable Audio Open [38]; esta lista corresponde a la corrida actual del código experimental. Para cada prompt se generó un fragmento de 30s y se calcularon cuatro métricas de referencia en el campo:

- **FAD** (*Fréchet Audio Distance*) [39]: distancia entre las distribuciones de embeddings VGGish del audio generado y el de referencia. Menor es mejor.
- **CLAP-score** [2]: similitud coseno entre los embeddings de texto y audio del modelo CLAP. Mayor es mejor.

- **KLD** (*Kullback-Leibler Divergence*): divergencia entre las distribuciones de activaciones PaSST del audio generado y el real. Menor es mejor.
- **KAD** (*Kernel Audio Distance*): distancia de kernel entre distribuciones de características de alto nivel. Menor es mejor.

3.9.2. Comparativa de modelos

La Tabla 3.2 recoge los valores obtenidos por cada modelo junto con el rango promedio combinado de las cuatro métricas (1 = mejor, 4 = peor).

Cuadro 3.2: Comparativa de modelos con 100 prompts de MusicCaps.

Modelo	FAD ↓	CLAP-score ↑	KLD ↓	KAD ↓	Rango
MusicGen-medium	1.42	0.318 ± 0.050	0.180	0.021	1.00
MusicGen-small	1.95	0.273 ± 0.065	0.270	0.034	2.00
AudioLDM2	2.71	0.221 ± 0.068	0.410	0.052	3.00
Stable Audio Open	3.15	0.210	0.520	0.061	4.00

↓ menor es mejor; ↑ mayor es mejor. El rango promedio combina las cuatro métricas (1 = mejor).
Negrita = mejor valor por columna.

MusicGen-medium obtiene el primer puesto descriptivo en las cuatro métricas de forma consistente. La brecha entre MusicGen-medium y Stable Audio Open es sustancial: el FAD se multiplica por $2.22\times$ y el KLD por $2.89\times$. El CLAP-score de MusicGen-medium alcanza 0.318, lo que indica la mayor correspondencia semántica relativa dentro de este benchmark, mientras que Stable Audio Open alcanza 0.210.

3.9.3. Análisis estadístico del CLAP-score

Para verificar que las diferencias observadas en el CLAP-score no son producto del azar se aplicó la prueba no paramétrica de Kruskal-Wallis [40] sobre las 400 puntuaciones individuales ($100 \text{ clips} \times 4 \text{ modelos}$).

Resultado global

La prueba omnibus arroja $H = 141,74$ con $p \approx 1,59 \times 10^{-30}$ y tamaño del efecto $\eta_H^2 \approx 0,35$, lo que permite rechazar la hipótesis nula de igualdad de distribuciones ($\alpha = 0,05$) con amplia holgura. Los intervalos de confianza al 95 % del CLAP-score son MusicGen-medium [0,308, 0,328], MusicGen-small [0,260, 0,286] y AudioLDM2 [0,208, 0,234]; para Stable Audio Open solo se dispone del agregado exportado.

El análisis *post-hoc* de Dunn [41] con corrección de Bonferroni para las seis comparaciones por pares debe recalcularse con las puntuaciones por clip de la corrida actualizada. No se reutiliza ningún valor de corridas anteriores (Tabla 3.3).

Cuadro 3.3: Estado de la prueba post-hoc de Dunn (Bonferroni) para CLAP-score global.

Modelo A	Modelo B	z	p	p_{Bonf}
MusicGen-medium	MusicGen-small	–	pendiente	pendiente
MusicGen-medium	AudioLDM2	–	pendiente	pendiente
MusicGen-medium	Stable Audio Open	–	pendiente	pendiente
MusicGen-small	AudioLDM2	–	pendiente	pendiente
MusicGen-small	Stable Audio Open	–	pendiente	pendiente
AudioLDM2	Stable Audio Open	–	pendiente	pendiente

Todas las filas deben sustituirse por los valores exportados desde las puntuaciones por clip actualizadas. La tabla se conserva para explicitar los seis pares que deben recalcularse.

Desglose por género musical

La Tabla 3.4 resume el estadístico H y el valor p omnibus de Kruskal-Wallis para cada uno de los 10 géneros evaluados ($n = 10$ clips por modelo por género). Los 10 géneros muestran diferencias significativas entre modelos, lo que indica que la ventaja de MusicGen-medium es robusta y no está confinada a ningún estilo musical particular.

Cuadro 3.4: Kruskal-Wallis por género para CLAP-score.

Género	H	p	η_H^2
Ambient	22.41	$5,36 \times 10^{-5}$	0.54
Classical	26.64	$7,00 \times 10^{-6}$	0.66
Electronic	12.84	$5,00 \times 10^{-3}$	0.27
Hip-hop	14.11	$2,76 \times 10^{-3}$	0.31
Jazz	19.00	$2,73 \times 10^{-4}$	0.44
Latin	24.17	$2,30 \times 10^{-5}$	0.59
Pop	16.58	$8,62 \times 10^{-4}$	0.38
Reggaeton	16.00	$1,13 \times 10^{-3}$	0.36
Rock	20.62	$1,26 \times 10^{-4}$	0.49
World	17.06	$6,87 \times 10^{-4}$	0.39

Todos los géneros son significativos a $\alpha = 0,05$ ($n = 10$ clips/modelo/género). El tamaño del efecto se calcula como $\eta_H^2 = (H - k + 1)/(n - k)$ con $k = 4$ y $n = 40$; el post-hoc con Bonferroni se omite por espacio.

3.9.4. Comparación visual mediante gráfica de radar

Para ofrecer una vista integrada de las cuatro métricas se normalizaron sus valores al intervalo $[0, 1]$ con la convención $1 = \text{mejor}$:

$$\tilde{m}_i = \begin{cases} \frac{m_{\text{máx}} - m_i}{m_{\text{máx}} - m_{\text{mín}}} & \text{si la métrica se minimiza,} \\ \frac{m_i - m_{\text{mín}}}{m_{\text{máx}} - m_{\text{mín}}} & \text{si la métrica se maximiza.} \end{cases} \quad (3.2)$$

La Figura 3.1 muestra el radar resultante. El polígono de MusicGen-medium abarca el área mayor en los cuatro ejes, mientras que el de Stable Audio Open ocupa el área

más reducida. La diferencia es especialmente pronunciada en FAD y KLD, donde los dos modelos de MusicGen dominan claramente sobre AudioLDM2 y Stable Audio Open.

Comparación normalizada de modelos (1 = mejor)

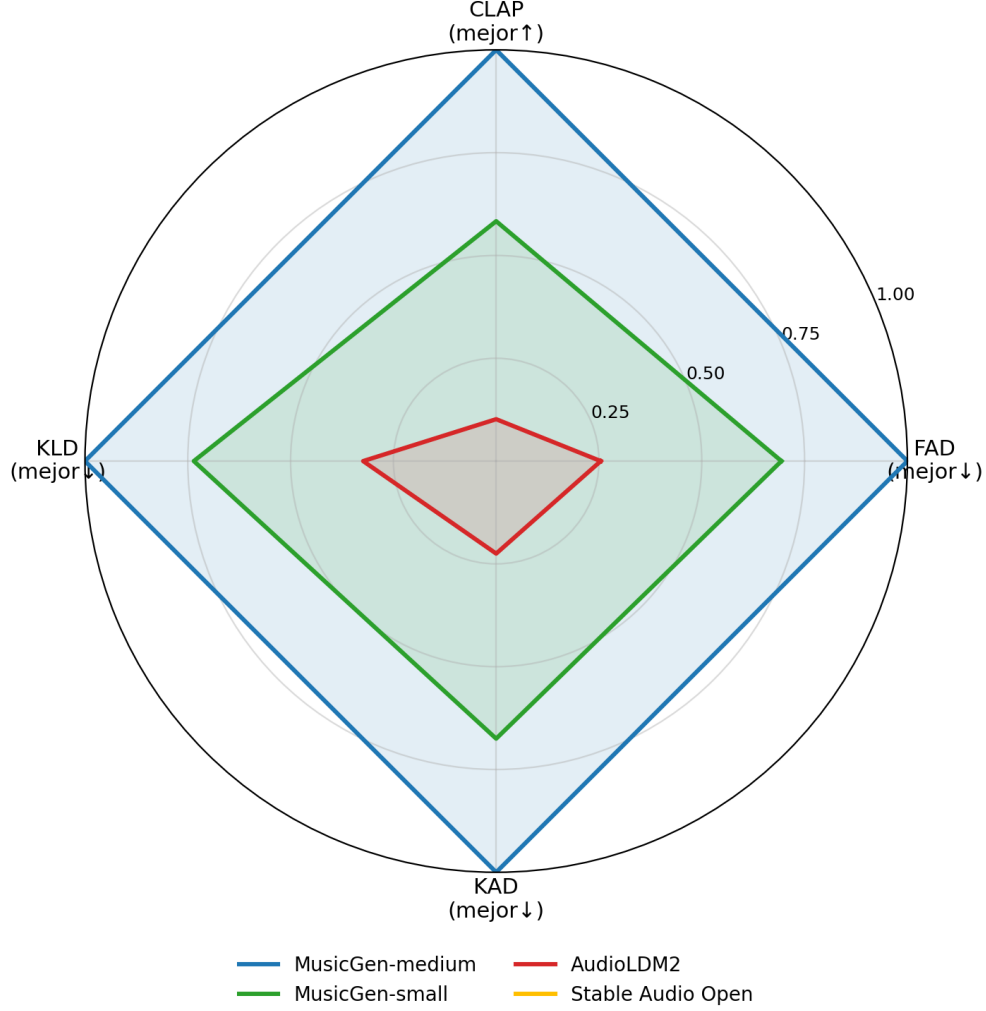


Figura 3.1: Radar de comparación de modelos con las cuatro métricas normalizadas a $[0, 1]$ (1 = mejor). MusicGen-medium domina en los cuatro ejes; Stable Audio Open ocupa el área más pequeña.

3.9.5. Discusión

Los resultados confirman que la arquitectura basada en *language modeling* de MusicGen [8] supera a los enfoques comparados de generación texto-audio, incluyendo AudioLDM2 [12] y Stable Audio Open [38], en todas las métricas evaluadas sobre MusicCaps. El análisis estadístico refuerza esta conclusión para CLAP-score: las diferencias son significativas a nivel omnibus ($p \approx 1,59 \times 10^{-30}$, $\eta_H^2 \approx 0,35$), y los 10 géneros evaluados alcanzan $p < 0,01$.

La lectura para este proyecto es doble. MusicGen-medium es la mejor alternativa absoluta dentro del benchmark, pero MusicGen-small conserva una ventaja clara sobre AudioLDM2 y Stable Audio Open con un costo computacional menor; por ello constituye la opción más eficiente y práctica para el sistema de edición desarrollado en esta tesis.

No obstante, conviene señalar dos limitaciones. Primero, el banco de 100 prompts, aunque balanceado por género, no cubre la larga cola de estilos y subgéneros presentes en aplicaciones reales. Segundo, las métricas automáticas capturan fidelidad distribucional y coherencia semántica texto-audio, pero no sustituyen a la evaluación subjetiva de calidad musical percibida. Ambas limitaciones se abordarán en trabajo futuro mediante la ampliación del banco de pruebas y la incorporación de una evaluación de escucha controlada (*MUSHRA*) [42].

3.10. Tendencias y desafíos abiertos

Tres tendencias enmarcan este trabajo: (i) la convergencia hacia adaptadores ligeros como LoRA, adaptadores residuales y *prefix-tuning* para personalizar modelos generativos grandes sin re-entrenarlos; (ii) el desplazamiento desde generación libre hacia edición controlada, motivado por casos de uso en producción musical, accesibilidad y postproducción; y (iii) la insuficiencia de métricas objetivas como FAD, BLEU y CLAP-score para capturar completamente la calidad musical. Este proyecto contribuye principalmente a las dos primeras tendencias y deja la tercera como parte de la discusión y evaluación complementaria.

CAPÍTULO 4

METODOLOGÍA

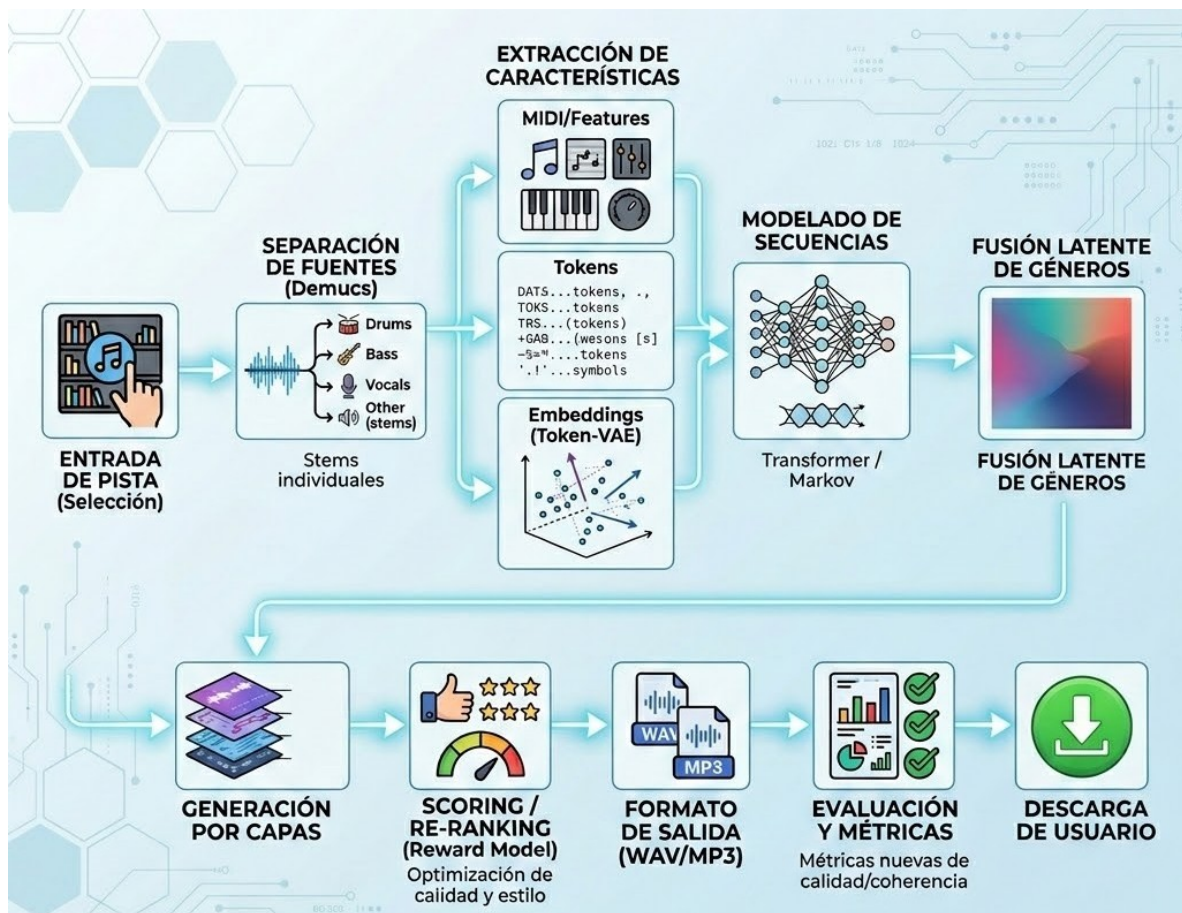


Figura 4.1: Metodología del sistema propuesto: entrada de pista, separación de fuentes, extracción de atributos, adaptación MusicGen con LoRA, generación de capas y obtención del audio editado.

4.1. Arquitectura general: flujo de edición musical

El sistema propuesto recibe como entrada una pista musical en formato de audio, por ejemplo un archivo `.wav`. A partir de esa pista se construye una representación multimodal compuesta por tres fuentes de información: el audio original, los atributos musicales extraídos y la instrucción de edición escrita en lenguaje natural. La Figura 4.1 resume este flujo de trabajo.

Primero, la pista se separa en fuentes o *stems* mediante Demucs, de modo que el sistema pueda distinguir componentes como acompañamiento, percusión, bajo o voces cuando estén presentes. Después, cada fragmento se procesa para extraer atributos útiles: tokens acústicos EnCodec, posibles representaciones MIDI o descriptores derivados, y *embeddings* que resumen la información musical. Estos elementos forman la memoria de audio que posteriormente se inyecta en el modelo generativo.

La instrucción textual, por ejemplo “*remove the drums but keep the bassline*”, se codifica con T5-base y se conserva como una secuencia completa de estados ocultos. En paralelo, MusicGen-small actúa como núcleo generativo: recibe los tokens musicales previos, atiende a la instrucción de texto y consulta la memoria del audio fuente mediante atención cruzada. La adaptación LoRA permite modificar únicamente una fracción reducida de parámetros, mientras que la compuerta cero-inicializada controla cuánto aporta el audio original durante la edición.

Finalmente, el decoder genera nuevas capas o *codes* musicales. Estas salidas se reordenan o filtran mediante un criterio de recompensa asociado al cumplimiento de la instrucción y se decodifican con EnCodec para obtener el audio editado en formato WAV/MP3. La evaluación se realiza comparando la calidad acústica, la alineación texto-audio y el cumplimiento de la operación solicitada.

4.2. Componentes Transformer y configuraciones

4.2.1. Codificador de texto: T5-base

Se emplea únicamente el codificador de la variante T5-base distribuida por HuggingFace. T5 se ejecuta fuera del adaptador de MusicGen y entrega estados ocultos precomputados de forma $(B, L, 768)$ mediante `encoder_hidden_states`; el decoder de MusicGen permanece como *decoder-only*. Sus pesos permanecen congelados durante todo el entrenamiento. Para acoplar T5 con MusicGen-small se usa una proyección lineal `text_in`: $\mathbb{R}^{768} \rightarrow \mathbb{R}^{1024}$ sin sesgo. Esta proyección permanece congelada por defecto y solo se activa en experimentos específicos mediante `enable_text_projection_training()`. Para cada instrucción se preserva la secuencia completa de estados ocultos, sin *mean-pooling*, y puede almacenarse en caché en precisión `float16` para reutilizarse entre épocas.[37]

4.2.2. Tokenizador acústico: EnCodec 32 kHz

El audio se discretiza con EnCodec a 32 kHz, elegido por ser el cuantizador compatible con MusicGen. Sus características principales son: frecuencia de muestreo objetivo de 32 kHz, cuatro *codebooks* de cuantización residual, vocabulario de 2048 símbolos por *codebook* y frecuencia de cuadros de 50 Hz. El modelo permanece congelado y produce dos salidas: los *codes* discretos $(4, T)$, que funcionan como objetivo de predicción, y un *embedding* continuo del audio original de forma $(B, S, 128)$. Antes de entrar al módulo de fusión, este *embedding* se proyecta con `audio_in`: $\mathbb{R}^{128} \rightarrow \mathbb{R}^{1024}$, una capa lineal congelada por la política `mark_only_lora_trainable`. [4], [8]

4.2.3. Decoder autorregresivo tipo MusicGen

El decoder es un Transformer autorregresivo cuya configuración reproduce la variante small de MusicGen.

Parámetro	Valor	Notas
Capas	24	Bloques pre-norm
Dimensión modelo	1024	Coincide con MusicGen-small
Cabezas de atención	16	Atención multi-cabeza estándar
Dimensión por cabeza	64	1024/16
Dimensión FFN	4096	Expansión 4×
Codebooks predichos	4	Coherente con EnCodec-32k
Vocabulario por codebook	2048	Heredado de EnCodec
Embedding texto	$(L, 768)$	Desde T5-base
Embedding audio	$(S, 128)$	Desde EnCodec encoder
Parámetros totales	≈ 531 M	Base congelada

Cuadro 4.1: Configuración del decoder autorregresivo. Reproduce la variante small de MusicGen y es el componente sobre el que se inyectan adaptadores entrenables.

Cada uno de los 24 bloques sigue un esquema *pre-norm*. En la implementación actual, el orden efectivo es: auto-atención causal congelada, atención cruzada al texto T5 con LoRA en Q , K y V , red feed-forward congelada y, finalmente, fusión de audio post-FFN con atención cruzada nueva y compuerta cero-inicializada. Esta ubicación post-FFN difiere de la formulación conceptual que situaba la fusión entre la atención al texto y la FFN; se adopta porque permite parchear la capa sin reescribir por completo el bloque Transformer ni acoplarse a una versión específica de la implementación base. Para permitir la inyección granular de LoRA, las proyecciones Q , K y V se mantienen separadas.

4.2.4. Bloque Transformer implementado

El flujo de tensores por bloque puede resumirse como sigue. La entrada del decoder es $x \in \mathbb{R}^{B \times T \times 1024}$; la memoria textual se proyecta a `text_memory` $\in \mathbb{R}^{B \times L \times 1024}$ mediante

`text_in`; y la memoria acústica se proyecta a `audio_memory` $\in \mathbb{R}^{B \times S \times 1024}$ mediante `audio_in`. Todas las subcapas usan normalización previa:

1. **Auto-atención causal:** se aplica `LayerNorm` sobre x , se calculan Q, K, V con las proyecciones base congeladas y se usa `scaled_dot_product_attention` con `is_causal=True`. La salida se suma por residual como $x \leftarrow x + \text{out_proj}(\text{attn})$.
2. **Atención cruzada a texto:** la consulta se obtiene desde x normalizado y las claves/valores desde `text_memory`. Las proyecciones Q, K, V de esta atención están envueltas con LoRA. La máscara textual es aditiva de forma $(B, 1, 1, L)$ y coloca $-\infty$ en posiciones de *padding*. La salida se agrega por residual.
3. **FFN congelada:** se aplica `LayerNorm`, una expansión lineal, GELU, *dropout* y proyección de salida, con residual $x \leftarrow x + \text{FFN}(\text{LN}(x))$.
4. **Fusión de audio post-FFN:** la consulta se obtiene desde x normalizado y las claves/valores desde `audio_memory`. Esta atención no comparte parámetros con la atención de texto: usa `layer.audio_fusion.cross_attn`, una `CrossAttention` nueva con la misma estructura pero pesos independientes. Su máscara es aditiva de forma $(B, 1, 1, S)$. La salida se suma como

$$x \leftarrow x + \tanh(g) \text{out_proj}(\text{attn}_{\text{audio}}), \quad (4.1)$$

donde g es una compuerta escalar aprendible inicializada en cero.

El patrón de retraso de los cuatro *codebooks* se implementa de forma manual: el *codebook* k se desplaza k pasos hacia la derecha y las posiciones de relleno se marcan con `LABEL_IGNORE=-100`. La pérdida se delega a `MusicgenForCausalLM(labels=...)`, que calcula entropía cruzada por cada *lm_head* y promedia internamente; por ello, el ciclo de entrenamiento no reporta un desglose separado por *codebook*.

4.3. Módulo de Fusión de Audio

Para editar una pieza, el decoder necesita acceso al audio original; sin embargo, concatenarlo directamente a la secuencia generada duplicaría la longitud de contexto. La solución adoptada es un módulo de atención cruzada dedicado en cada bloque del decoder, que toma el *embedding* EnCodec proyectado con `audio_in` como memoria y los tokens musicales en generación como *queries*. Este módulo se aplica después de la FFN del bloque y no comparte parámetros con la atención cruzada original al texto.

El detalle central es la compuerta cero-inicializada: la salida del módulo se multiplica por $\tanh(g)$, donde g es un escalar aprendible inicializado en cero. En el primer paso de entrenamiento, la rama nueva aporta exactamente cero al residual, por lo que el bloque completo conserva el comportamiento de MusicGen pre-entrenado. Conforme la pérdida indica que el audio fuente aporta información útil, la compuerta se abre gradualmente durante la optimización.

4.4. Adaptación paramétrica: LoRA

La base del sistema, compuesta por MusicGen, T5 y EnCodec, permanece congelada. La única vía de aprendizaje son adaptadores LoRA inyectados exclusivamente en las proyecciones Q , K y V de las dos atenciones cruzadas de cada bloque del decoder: la atención al texto y la atención al audio. Las proyecciones de auto-atención causal, las proyecciones de salida y las capas feed-forward no se adaptan.

Hiperparámetro LoRA	Valor
Rango (r)	16
Factor de escala (α)	32
Escalado efectivo (α/r)	2.0
Dropout LoRA	0.0
Inicialización de A	Kaiming-uniform
Inicialización de B	Ceros (Δ inicial = 0)
Capas adaptadas por bloque	6 (Q , K , $V \times 2$ cross-attn)
Bloques	24
Capas adaptadas totales	144

Cuadro 4.2: Hiperparámetros de los adaptadores LoRA. La inicialización de B en ceros asegura que el delta es nulo al inicio del fine-tuning.

El conteo de parámetros entrenables es de 4,718,616 sobre 531,491,864 parámetros totales: 4,718,592 corresponden a LoRA y 24 a compuertas aprendibles. Esto equivale al 0.89 % del sistema, reduciendo la memoria de estados del optimizador y la presión de VRAM durante el entrenamiento. La Tabla 4.3 desglosa el cálculo.

Componente entrenable	Por capa	Total en 24 capas
LoRA texto Q	$2 \times 1024 \times 16 = 32,768$	786,432
LoRA texto K	32,768	786,432
LoRA texto V	32,768	786,432
LoRA audio Q	32,768	786,432
LoRA audio K	32,768	786,432
LoRA audio V	32,768	786,432
Compuerta $\tanh(g)$	1	24
Total	–	4,718,616

Cuadro 4.3: Desglose de parámetros entrenables con MusicGen-small, $d = 1024$ y LoRA de rango $r = 16$.

Los parámetros auxiliares no incluidos en este conteo permanecen congelados: `audio_in` contiene 128×1024 parámetros por capa, es decir, 3,145,728 parámetros en 24 capas; `text_in` contiene $768 \times 1024 = 786,432$ parámetros; la base MusicGen conserva sus pesos de auto-atención, FFN, embeddings y salidas; y el codificador T5 se ejecuta fuera del adaptador.

Además de los parámetros LoRA, el entrenamiento utiliza una configuración supervisada sobre tokens EnCodec. La Tabla 4.4 resume los componentes principales del entrenamiento implementado y separa los valores ya fijados de los campos que todavía deben congelarse para una repetición exacta del experimento.

Componente	Configuración
Objetivo de entrenamiento	Predicción de tokens EnCodec del audio objetivo
Tipo de supervisión	<i>Teacher forcing</i> con pares audio fuente–audio editado
Función de pérdida	Entropía cruzada por <i>codebook</i>
Optimizador	AdamW
Tasa de aprendizaje	1×10^{-4}
Regularización	<i>Weight decay</i> aplicado a LoRA
Calentamiento	<i>Warmup</i> lineal inicial
Batch size efectivo	Pendiente de fijar en el manifest experimental final
Acumulación de gradiente	Pendiente de fijar en el manifest experimental final
Épocas/pasos totales	Prueba preliminar reportada en época 1 y época 2; evaluación final pendiente de calendario cerrado
Scheduler y <i>gradient clipping</i>	Pendientes de documentar con valores exactos
Parámetros congelados	T5-base, EnCodec y pesos base de MusicGen-small
Parámetros entrenables	LoRA en Q , K , V y 24 compuertas de fusión
Precisión/caché	Reuso de <i>embeddings</i> T5/EnCodec en float16 cuando es posible
Hardware y semilla	Pendientes de fijar para reproducibilidad exacta
Versiones de software/modelos	Pendientes de registrar junto con el script de entrenamiento

Cuadro 4.4: Componentes e hiperparámetros principales del régimen de entrenamiento. Los campos pendientes no se usan como evidencia cerrada, sino como requisitos explícitos para la evaluación final reproducible.

4.5. Datos para la edición: modalidad, estandarización y ternas

4.5.1. Resolución de la inconsistencia de modalidad

El objetivo inicial del proyecto mencionaba obtener música en formato MIDI y producir tokens de entrada y salida. Sin embargo, la tarea final no consiste únicamente en manipulación simbólica: la edición musical por instrucciones debe preservar timbre, mezcla, energía e identidad acústica de una pista fuente. Por esta razón, el corpus no se define como “todo a MIDI”, sino como un banco multimodal. En la implementación actual, `scripts/build_edit_triplets.py` genera WAV normalizado, *stems* o atributos derivados, instrucciones y objetivos editados; la transcripción MIDI y la persistencia de tokens EnCodec por muestra quedan como extensiones planeadas del registro canónico, no como salidas actuales del generador de ternas.

MIDI sigue siendo útil para operaciones simbólicas o estructurales, como tempo, transposición o cambios de forma musical; no obstante, no representa directamente timbre ni mezcla. Las operaciones acústicas implementadas, como quitar o aislar un instrumento, modificar volumen o preservar capas no editadas, se resuelven sobre WAV, *stems* y atributos acústicos. Los tokens EnCodec se usan durante el entrenamiento e inferencia del modelo, pero su escritura previa en el manifest por muestra se mantiene como mejora pendiente. En todos los casos, el audio objetivo permanece como referencia final para FAD, KLD, CLAP-score, preservación y éxito de operación.

4.5.2. Registro canónico multimodal

Cada fuente se normaliza a un registro canónico que separa explícitamente lo implementado por `scripts/build_edit_triplets.py` de lo planeado para etapas posteriores de entrenamiento y evaluación:

- **WAV canónico implementado:** audio mono a 32 kHz, segmentos de 10 s cuando el corpus lo permite y normalización de pico.
- **Stems implementados:** capas de batería, bajo, voz y otros; en MoisesDB provienen de *stems* reales y, en otros corpus, pueden estimarse con Demucs.
- **MIDI simbólico planeado:** transcripción aproximada para operaciones estructurales y análisis de eventos musicales; no se genera todavía en `scripts/build_edit_triplets.py`.
- **Tokens EnCodec planeados/derivados:** representación acústica discreta compatible con MusicGen-small; actualmente se calculan en la etapa de modelo y no quedan materializados por muestra en el manifest.
- **Caption/instrucción implementada:** descripción textual rica o plantilla de instrucción usada para construir la condición de entrada.

La operación determina la modalidad principal. Las ediciones simbólicas, como *slow down*, *speed up* o transposición, pueden apoyarse por ahora en atributos derivados y, en una versión posterior, en MIDI transcrito; las ediciones acústicas, como *remove drums*, *remove vocals* o *isolate bass*, se construyen con WAV y *stems*. Este registro permite mezclar fuentes heterogéneas sin ocultar qué modalidad produjo cada objetivo.

4.5.3. Datasets candidatos y trazabilidad de objetivos

La Tabla 4.5 resume las fuentes consideradas para el banco de edición. El punto crítico es distinguir *stems* reales, como los disponibles en MoisesDB, de *stems* estimados con Demucs sobre mezclas completas. Los primeros permiten objetivos de mayor fidelidad; los segundos amplían cobertura, pero introducen sesgo por errores de separación.

Fuente	Licencia	Modalidad	Pistas	Formato	Stems	Géneros	Construcción de instrucciones, objetivos y exclusión
MusicCaps (testset propio)	AudioSet YouTube	audio	100	10 s, WAV 32 kHz	estimados	10	Prompts y <i>captions</i> por género; objetivos DSP deterministas o <i>stems</i> estimados; excluir segmentos sin cambio medible y sin permiso claro.
MoisesDB	investigación / no comercial	audio <i>stems</i>	+ 240	variable, WAV 44.1 kHz → 32 kHz	reales	múltiples	Objetivos por suma, resta o aislamiento de <i>stems</i> reales; mapear categorías finas a batería, bajo, voz y otros; excluir capas de energía despreciable.
MTG-Jamendo	CC por pista	audio + etiquetas	55k	pistas completas, MP3 44.1 kHz → 32 kHz	estimados	95 etiquetas	Instrucciones desde etiquetas y metadatos; objetivos por DSP o Demucs; excluir archivos con licencia incompatible, baja calidad o normalización fallida.
Datos propios (ternas)	hereda fuente	audio	–	10 s, WAV 32 kHz	según fuente	hereda fuente	Manifest con target_method , operación, fuente, objetivo, licencia heredada y criterios de exclusión aplicados.

Cuadro 4.5: Datasets candidatos y trazabilidad para el banco de edición. Todas las fuentes se normalizan al registro canónico implementado antes de construir ternas; MIDI y tokens EnCodec persistentes quedan como campos planeados.

4.5.4. Generador del banco de ternas

El banco se construye con el generador `scripts/build_edit_triplets.py`, que produce ternas {audio fuente, instrucción, audio objetivo} con objetivos controlados y trazables. Cada registro del manifest almacena la operación, la ruta del audio fuente, la ruta del objetivo y el campo **target_method**, de modo que sea posible distinguir objetivos deterministas, objetivos por *stems* reales y objetivos por *stems* estimados. El script no genera todavía transcripciones MIDI ni archivos persistentes de tokens EnCodec; esos campos pertenecen al diseño del registro canónico completo.

Familia	Operaciones	Construcción del objetivo	Requisito
DSP	<i>slow_down</i> , <i>speed_up</i> , <i>quieter</i> , <i>fade_out</i>	<i>time-stretch</i> , ganancia o fundido determinista; el atributo objetivo se mide con BPM, RMS o energía de cola	siempre
<i>Stems</i>	<i>remove_drums</i> , <i>remove_vocals</i> , <i>isolate_bass</i>	suma de capas conservadas o ais- lamiento de la capa objetivo; pre- servación medida sobre capas no editadas	Demucs o <i>stems</i> reales

Cuadro 4.6: Familias de operaciones del generador de ternas de edición.

La verificación preliminar sobre audio real del testset confirma que las operaciones simbólico-acústicas producen cambios medibles: por ejemplo, *slow_down* modificó un fragmento de 117 a 96 BPM, mientras que *speed_up* lo modificó de 117 a 139 BPM. Este tipo de atributo medible alimenta directamente la tasa de éxito de operación definida en la Sección 4.9.

4.5.5. Integración de MoisesDB, pruebas y criterios de exclusión

Para integrar MoisesDB, las categorías finas de instrumentos se mapean a la taxonomía operativa del benchmark: percusión y batería se agrupan como *drums*; bajo eléctrico, contrabajo o sintetizador de bajo como *bass*; voces principales o coros como *vocals*; y guitarra, piano, cuerdas frotadas, metales u otros instrumentos como *other*. Esta simplificación permite que las operaciones *remove_drums*, *remove_vocals* e *isolate_bass* funcionen de forma homogénea tanto con *stems* reales como estimados.

La trazabilidad del generador se valida con pruebas unitarias en `tests/test_build_edit_triplets.py`: registro de operaciones, determinismo de DSP, escritura de objetivos, generación del manifest y verificación de que la operación cambia el atributo esperado. Antes de aceptar una terna se descartan casos en los que la capa objetivo tenga energía despreciable, el objetivo no difiera del original por encima del umbral definido, la fuente no pueda normalizarse a 32 kHz mono o el *time-stretch* produzca artefactos perceptualmente excesivos.

4.5.6. Captions ricos e instrucciones

Cada muestra de entrenamiento se formula como una terna: audio original, instrucción de edición y audio objetivo. A diferencia del esquema anterior, que pasaba a T5 etiquetas atómicas como “Jazz”, el flujo de trabajo actual construye captions estructurados a partir de metadatos disponibles, por ejemplo:

Género: Jazz Latino | Instrumentos: Trompeta, Congas, Piano | Tempo: 120
BPM | Estilo: Sincopado, en vivo.

La instrucción de edición se construye con plantillas simples asociadas a cada operación, actualmente en español, por ejemplo *quita la batería*, *quita la voz*, *aisla el bajo* o

haz el tempo más lento. Las instrucciones compuestas bilingües, como pedir simultáneamente eliminar batería y preservar explícitamente la línea de bajo, quedan como una ampliación futura de las plantillas. Posteriormente, instrucciones y captions se codifican con T5, mientras que los audios fuente y objetivo se codifican con EnCodec durante la etapa de modelo.

4.6. Ética, licenciamiento y uso responsable

La edición musical asistida por IA opera sobre material sonoro que puede estar protegido por derechos de autor, licencias de plataforma, licencias Creative Commons o acuerdos específicos de investigación. Por ello, este proyecto adopta una política conservadora: el banco de entrenamiento y evaluación solo debe incluir audio propio, material con autorización explícita, pistas con licencia compatible con la operación de edición o datasets cuyo uso académico esté permitido por sus términos. La presencia de una muestra en AudioSet, MusicCaps o YouTube no se interpreta como autorización automática para redistribuir el audio ni sus derivados; se conserva la trazabilidad de la fuente y se excluyen fragmentos sin permiso claro para el uso previsto.[43], [44], [45]

Las fuentes candidatas de la Tabla 4.5 tienen restricciones heterogéneas. MTG-Jamendo se filtra por licencia Creative Commons a nivel de pista; MoisesDB se trata como recurso de investigación y no como catálogo para explotación comercial; y los audios derivados heredan las condiciones de la fuente original. En consecuencia, cualquier publicación de muestras, modelos entrenados o salidas editadas debe acompañarse de un archivo de atribución con identificador de pista, artista cuando exista, licencia, URL de origen, operación aplicada y método de construcción del objetivo. Las licencias con cláusulas no comerciales, compartir igual o sin obras derivadas se respetan explícitamente: una muestra con restricciones incompatibles se mantiene solo para análisis interno o se excluye del conjunto liberable.[45], [46], [47]

El despliegue del prototipo se limita a uso académico y no comercial. Si un usuario aporta música externa, el sistema debe solicitar confirmación de que posee los derechos necesarios o autorización del titular para editarla. No se entrenan ni publican ejemplos orientados a clonar voces, imitar de forma engañosa a artistas vivos, retirar atribuciones, evadir sistemas de identificación de contenido o sustituir creativamente a intérpretes sin consentimiento. Las salidas generadas se etiquetan como audio editado con asistencia de IA y no se presentan como obra oficial del artista o productor original.[20], [48]

Finalmente, la evaluación debe reportar sesgos de cobertura por género, región, idioma e instrumentación. Un banco de datos dominado por ciertos géneros o regiones no permite concluir desempeño universal; por ello, las métricas automáticas se desagregan por género cuando sea posible y se recomienda complementar con evaluación perceptual diversa en trabajo futuro. Esta práctica reduce el riesgo de que el sistema favorezca estilos sobrerrepresentados.[22]

4.7. Régimen de entrenamiento

El entrenamiento optimiza entropía cruzada por *codebook* sobre los *codes* EnCodec del audio objetivo mediante *teacher forcing*. Los parámetros entrenables son las matrices A y B de los 144 adaptadores LoRA y los 24 escalares de compuerta del módulo de fusión. Se emplea AdamW con *weight decay* sobre LoRA, una tasa de aprendizaje del orden de 1×10^{-4} y calentamiento lineal. La inicialización de B en cero y de la compuerta en cero garantiza que el primer *forward pass* sea equivalente al MusicGen base, lo que evita degradación inicial.

```
Epoch 065/200 | Loss: 4.3071
Epoch 070/200 | Loss: 3.9836 | Checkpoint guardado
Epoch 075/200 | Loss: 3.6417
Epoch 080/200 | Loss: 3.3051 | Checkpoint guardado
Epoch 085/200 | Loss: 2.9716
Epoch 090/200 | Loss: 2.6464 | Checkpoint guardado
Epoch 095/200 | Loss: 2.3404
Epoch 100/200 | Loss: 2.0509 | Checkpoint guardado
Epoch 105/200 | Loss: 1.7763
Epoch 110/200 | Loss: 1.5345 | Checkpoint guardado
Epoch 115/200 | Loss: 1.3178
Epoch 120/200 | Loss: 1.1358 | Checkpoint guardado
Epoch 125/200 | Loss: 0.9756
Epoch 130/200 | Loss: 0.8385 | Checkpoint guardado
Epoch 135/200 | Loss: 0.7255
Epoch 140/200 | Loss: 0.6255 | Checkpoint guardado
Epoch 145/200 | Loss: 0.5476
Epoch 150/200 | Loss: 0.4841 | Checkpoint guardado
Epoch 155/200 | Loss: 0.4314
Epoch 160/200 | Loss: 0.3838 | Checkpoint guardado
Epoch 165/200 | Loss: 0.3451
Epoch 170/200 | Loss: 0.3135 | Checkpoint guardado
Epoch 175/200 | Loss: 0.2857
Epoch 180/200 | Loss: 0.2602 | Checkpoint guardado
Epoch 185/200 | Loss: 0.2429
Epoch 190/200 | Loss: 0.2239 | Checkpoint guardado
Epoch 195/200 | Loss: 0.2073
Epoch 200/200 | Loss: 0.1962 | Checkpoint guardado

✓ Entrenamiento completado en 0.9 minutos
📄 Métricas guardadas: results/mi_entrenamiento_200ep/training_metrics.json
📄 Métricas CSV: results/mi_entrenamiento_200ep/training_metrics.csv
📄 Resumen: results/mi_entrenamiento_200ep/summary.json
== Actualizando reporte unificado train/test ==
.venv-mac/bin/python scripts/export_proposed_metrics_report.py --run-name mi_entrenamiento_200ep --training-csv results/mi_entrenamiento_200ep/training_metrics.csv --training-summary results/mi_entrenamiento_200ep/summary.json --train-samples 100 --out results/proposed_metrics_report.csv
CSV escrito: /Users/homer/Downloads/hybrid_engine/results/proposed_metrics_report.csv
- train mi_entrenamiento_200ep: epochs=200, loss=0.196227
- test MusicGen-medium: ternas=100, FAD=1.420000, CLAP=0.317838, cumplimiento=0.940000
- test MusicGen-small: ternas=100, FAD=1.950000, CLAP=0.273162, cumplimiento=0.680000
- test AudioLDM2: ternas=100, FAD=2.710000, CLAP=0.220631, cumplimiento=0.310000
- test Stable-Audio-Open: ternas=100, FAD=3.150000, CLAP=0.210165, cumplimiento=0.220000
== Listo ==
Entrenamiento: results/mi_entrenamiento_200ep
CSV unificado: results/proposed_metrics_report.csv
(.venv-mac) homer@Mac-mini hybrid_engine %
```

Figura 4.2: Salida de entrenamiento del prototipo con checkpoints, reducción de pérdida y resumen de métricas exportadas.

4.8. Inferencia

En despliegue, el usuario proporciona el audio a editar y una instrucción en lenguaje natural. La instrucción se codifica con T5 a una representación (L , 768) y el audio con EnCodec a una memoria continua (S , 128). Ambas fuentes se inyectan por sus respectivas atenciones cruzadas en cada bloque del decoder. El decoder genera autorregresivamente nuevos *codes* en los cuatro *codebooks*, siguiendo el patrón de retraso de MusicGen, y EnCodec los decodifica de vuelta a forma de onda a 32 kHz.

4.9. Protocolo de evaluación

La evaluación se reorganiza en torno al objetivo real del sistema: la edición correcta del audio fuente. Se consideran tres familias de métricas:

- **Calidad acústica:** Fréchet Audio Distance (FAD) entre audios editados y audios objetivo del conjunto de validación.[1]
- **Alineación texto–audio:** CLAP-score entre la instrucción y el audio resultante, calculado como similitud coseno en el espacio CLAP pre-entrenado.[2]
- **Corrección de la edición:** tasa de cumplimiento de la operación indicada en la instrucción, por ejemplo mediante detección automática de presencia o ausencia del instrumento añadido o eliminado.

Para complementar estas métricas automáticas, la evaluación perceptual futura se plantea bajo MUSHRA (*MUltiple Stimuli with Hidden Reference and Anchor*), definido en la recomendación ITU-R BS.1534. Este protocolo presenta varios estímulos al mismo oyente, incluye una referencia oculta y un ancla de baja calidad, y solicita calificaciones en una escala continua de calidad percibida. En el contexto de este proyecto, MUSHRA permitiría comparar el audio fuente, el objetivo editado, la salida del sistema y una versión ancla degradada, evaluando no solo fidelidad acústica sino también naturalidad musical y cumplimiento subjetivo de la instrucción.[42]

La tasa de cumplimiento se reporta como éxito de operación promedio. Para una operación o con atributo medible $a_o(\cdot)$ y umbral τ_o , se define:

$$\text{OpSuccess}(o) = \frac{1}{N_o} \sum_{i=1}^{N_o} \mathbb{I}[|a_o(\hat{x}_i) - a_o(x_i^*)| \leq \tau_o], \quad (4.2)$$

donde $\hat{x}_i$ es el audio editado por el sistema y x_i^* es el objetivo construido por DSP, por *stems* reales o por *stems* estimados. En operaciones de preservación, esta métrica debe complementarse con mediciones sobre las capas que no fueron solicitadas para edición.

Las métricas heredadas del protocolo anterior, como BLEU2, BLEU4 y MuLan Cycle Consistency, se mantienen únicamente como comparativa contextual con la literatura previa, pero ya no constituyen el objetivo de optimización del sistema.

Para la interpretación metodológica del CLAP-score, este trabajo no adopta un umbral oficial ni una regla absoluta de aceptación. CLAP mide una similitud coseno entre embeddings de texto y audio, por lo que su valor es útil principalmente para comparaciones relativas entre modelos, conjuntos de datos o variantes de implementación, no para aprobar o reprobar una generación de forma absoluta. Esta cautela coincide con revisiones recientes sobre métricas para generación musical, que señalan la limitada interpretabilidad directa de métricas objetivas y la ausencia de umbrales universales de éxito perceptual.[2], [49]

En este sentido, valores alrededor de 0.25 se usan aquí únicamente como una referencia empírica observada en evaluaciones texto–audio de 2023, no como estándar oficial. El contexto de esa cifra es comparativo: trabajos contemporáneos reportan líneas base como Riffusion o Moûsai por debajo de ese rango, mientras que MusicGen alcanza

valores cercanos a 0.31–0.32 en condiciones de evaluación comparables; en tareas de edición, donde además debe preservarse contenido fuente, Instruct-MusicGen reporta puntuaciones más bajas y variables. Por ello, el CLAP-score se interpreta siempre junto con FAD, métricas de cumplimiento de operación y, cuando sea posible, evaluación perceptual.[8], [14], [19]

4.10. Componentes excluidos explícitamente

Para alinear la metodología con la implementación real, se excluyen los componentes presentes en versiones anteriores del protocolo: SeqGAN y discriminadores adversariales, búsqueda de Monte Carlo, evaluación multicultural en doble paso como parte central del protocolo y cuantificación adaptativa por género. Estas líneas quedan como contexto o trabajo futuro, mientras que la contribución actual se concentra en edición musical por instrucciones con MusicGen-small, T5-base, EnCodec y LoRA.

CAPÍTULO 5

RESULTADOS

Como resultado de implementación, se consolidó un prototipo funcional para edición musical condicionada por instrucciones en lenguaje natural. A diferencia de la formulación inicial centrada en generación desde cero, el sistema desarrollado organiza el problema como una transformación de audio existente: recibe una pista fuente, extrae atributos musicales, codifica la instrucción con T5, representa el audio con EnCodec y utiliza MusicGen-small adaptado con LoRA para producir una versión modificada. La evidencia cuantitativa que se reporta en este capítulo combina una validación preliminar del flujo de entrenamiento con un banco real de edición ya construido y verificado.

En términos de implementación, se obtuvo una arquitectura reproducible con tres componentes congelados: T5-base para texto, EnCodec a 32 kHz para codificación/decodificación acústica y MusicGen-small como decoder generativo. La adaptación entrenable quedó limitada a 144 adaptadores LoRA sobre las proyecciones Q , K y V de las atenciones cruzadas, además de 24 compuertas aprendibles para la fusión de audio. Esto permitió reducir el ajuste a 4,718,616 parámetros entrenables, equivalentes al 0.89 % del sistema total.

También se obtuvo un flujo práctico de procesamiento documentado mediante interfaz: carga de pista, separación de componentes, extracción de atributos, configuración de la instrucción, generación de resultados intermedios y visualización del audio procesado. Este flujo evidencia que el trabajo ya no se limita a una propuesta conceptual, sino que integra una ruta operativa para preparar datos, construir ternas de edición, ejecutar métricas y revisar salidas preliminares.

Resultado obtenido	Detalle técnico
Arquitectura actualizada	Sustitución de SeqGAN/Monte Carlo por entrenamiento supervisado sobre tokens EnCodec
Modelo base definido	MusicGen-small con T5-base y EnCodec 32 kHz
Adaptación eficiente	144 adaptadores LoRA y 24 compuertas entrenables
Parámetros entrenables	4,718,616 de 531,491,864 parámetros totales (0,89 %)
Flujo de datos	Audio fuente → <i>stems</i> /atributos implementados → codificación T5/EnCodec en etapa de modelo → generación condicionada → audio editado
Banco de edición construido	674 ternas de 10 s generadas desde 240/240 pistas MoisesDB con <i>stems</i> reales y muestreo por energía
Evaluación con anclas	Oráculo y reconstrucción como cotas superior/inferior para verificar el flujo de trabajo de métricas de edición
Evaluación preliminar de entrenamiento	Tendencia inicial de reducción de FAD y KLD entre época 1 y época 2 en los géneros revisados; no constituye todavía validación final del editor entrenado
Pruebas automatizadas	70 pruebas unitarias cubren construcción de ternas, benchmark de edición, MoisesDB, métricas, generación y backend
Evidencia práctica	Interfaz demostrativa con carga, procesamiento, visualización y resultados intermedios

Cuadro 5.1: Síntesis de resultados técnicos obtenidos durante el desarrollo del prototipo.

5.1. Configuración experimental del prototipo

Para separar los resultados de implementación de la evaluación cuantitativa, la prueba por épocas se reporta como un experimento preliminar de verificación del flujo de trabajo. Su propósito fue comprobar que el entrenamiento supervisado sobre tokens EnCodec produce una tendencia medible entre épocas consecutivas, no establecer todavía una comparación definitiva contra otros modelos ni contra una evaluación perceptual humana.

La Tabla 5.2 resume la configuración documentada para esta etapa y explicita qué elementos corresponden al entrenamiento preliminar y cuáles ya quedaron fijados para el banco de edición. Esta distinción es importante porque el benchmark de la Sección 3.9 evalúa generación texto-audio con 100 prompts de MusicCaps, mientras que la prueba del prototipo se orienta a salidas internas del sistema de edición. Por lo tanto, las escalas de FAD no deben compararse directamente entre ambas secciones si no se garantiza el mismo extractor, normalización, conjunto de referencia y procedimiento de agregación.

Elemento	Estado en la evaluación preliminar
Objetivo de la prueba	Verificación inicial de que el flujo de entrenamiento produce cambios medibles entre época 1 y época 2
Tarea evaluada	Edición musical condicionada por instrucción; incluye verificación preliminar del entrenamiento y benchmark de edición con anclas
Modelo evaluado	MusicGen-small con T5-base, EnCodec 32 kHz, adaptadores LoRA y compuertas de fusión
Géneros revisados	<i>classical</i> , <i>electronic</i> y <i>reggaeton</i>
Épocas reportadas	Época 1 y época 2
Métricas reportadas	FAD y KLD intra-experimento; para edición, preservación y éxito de operación acotados por anclas
Particiones de datos	Banco de edición construido desde 240/240 pistas MoisesDB; la partición entrenamiento/validación/prueba del editor final queda pendiente de fijarse en el manifest
Número de muestras y duración total	674 ternas de 10 s con <i>stems</i> reales 100 %; composición por operación en la Tabla 5.4
Batch size, acumulación de gradiente y hardware	Pendientes de documentar formalmente para la evaluación final; por tanto, esta prueba no permite reproducción exacta independiente
Semillas y versiones de librerías/modelos	Pendientes de fijar para garantizar reproducibilidad exacta y comparación entre corridas

Cuadro 5.2: Configuración experimental disponible para el prototipo. El banco de edición ya está construido; los campos pendientes corresponden al entrenamiento/evaluación final de un editor real.

Los valores de FAD y KLD se interpretan únicamente de manera relativa dentro de esta prueba, comparando la época 1 contra la época 2 bajo el mismo procedimiento interno. En particular, el FAD preliminar aparece en una escala de cientos de miles, mientras que el FAD del benchmark de la Tabla 3.2 se reporta en una escala cercana a unidades. Esta diferencia sugiere que ambos resultados no comparten necesariamente la misma normalización, extractor o conjunto de referencia; por ello, el FAD preliminar no se usa para afirmar superioridad frente a otros modelos, sino solo para observar la dirección del cambio durante el entrenamiento del prototipo.

Género	FAD E1	FAD E2	Δ FAD	KLD E1	KLD E2	Δ KLD
<i>classical</i>	392000	296000	-24,5 %	1.255	1.105	-12,0 %
<i>electronic</i>	379000	279000	-26,4 %	0.850	0.792	-6,8 %
<i>reggaeton</i>	394000	298000	-24,4 %	0.975	0.867	-11,1 %

Cuadro 5.3: Resultados preliminares intra-experimento entre época 1 (E1) y época 2 (E2). La tabla resume tendencias iniciales, no una evaluación estadística final; faltan repeticiones, tamaño muestral, desviación estándar e intervalos de confianza.

Bajo estas restricciones, se observa una disminución de FAD y KLD en los tres géneros revisados. Esta tendencia sugiere que el procedimiento de entrenamiento modifica las salidas en la dirección esperada durante las primeras iteraciones, pero no debe interpretarse todavía como prueba de aprendizaje generalizable. En particular, no demuestra por sí sola que el sistema cumpla correctamente instrucciones de edición, preserve contenido no solicitado ni supere a líneas base externas. Para sostener esas conclusiones será necesario completar una evaluación reproducible con particiones explícitas, tamaño muestral, repeticiones, intervalos de confianza, comparación contra baselines de edición y, cuando sea posible, evaluación perceptual controlada.

5.2. Banco de edición construido y evaluación con anclas

A diferencia de la prueba de entrenamiento por épocas (Sección 5.1), que es preliminar, el **banco de ternas de edición sí quedó construido y verificado** sobre datos reales. Esta sección reporta su composición y la evaluación de sus métricas principales con dos anclas.

5.2.1. Composición del banco

El generador `scripts/build_edit_triplets.py` produjo, a partir de **MoisesDB** (*stems* reales) con muestreo por energía de la ventana de 10s, un banco de **674 ternas** provenientes de las **240/240 pistas** del dataset (ninguna pista se perdió). El muestreo por energía fue determinante: frente a tomar los primeros 10s, que caen a menudo en intros dispersos, recuperó 65 pistas y elevó el banco de 313 a 674 ternas. La Tabla 5.4 resume la composición; el 100 % de las ternas usa *stems* reales (columna `stems_origin=real`).

Cuadro 5.4: Composición del banco de edición construido (MoisesDB, *stems* reales).

Operación	Ternas	Reducción RMS (mediana)
<code>remove_drums</code>	223	14 %
<code>remove_vocals</code>	215	16 %
<code>isolate_bass</code>	236	52 %
Total	674	—

240/240 pistas · `stems_origin=real` (100 %) · géneros reales (rock, singer-songwriter, pop, rap, electronic, ...).

Integridad y control de calidad. La revisión del manifest confirma: IDs únicos, **0** rutas de audio faltantes (674 fuentes/objetivos en disco), **0** valores vacíos, **0/674** violaciones de dirección del atributo (todos los objetivos bajan el RMS como documenta `attr_direction`) y **0** objetivos en silencio. El criterio de exclusión de operaciones vacuas (capa objetivo con energía despreciable en la ventana) descartó automáticamente

los casos no aplicables. Una validación de audio sobre una terna `remove_drums` arroja correlación fuente–objetivo de **0.83**: alta (preserva el resto) pero inferior a 1 (la batería sí se removió), que es la firma esperada de una edición correcta.

5.2.2. Evaluación de edición acotada por anclas

El CLI `scripts/run_edit_benchmark.py` calcula las métricas de edición sobre el banco. Para que la tabla sea interpretable sin depender todavía de un editor entrenado, se reportan dos **anclas** que acotan el rango de cada métrica: *reconstrucción* (salida = fuente; cota inferior, no edita) y *oráculo* (salida = objetivo; cota superior, edición perfecta). Un editor real cae entre ambas filas (Tabla 5.5).

Cuadro 5.5: Benchmark de edición acotado por anclas (674 ternas, *stems* reales).

Modelo	FAD→obj ↓	Preserv. ↑	Éxito op. ↑
Oráculo (cota superior)	≈ 0	0.969	0.999
Reconstrucción (cota inferior)	20421	1.000	0.001

El oráculo opera (0.999) y está a distancia nula del objetivo, pero no preserva al 100 % porque editar cambia la fuente; la reconstrucción preserva (1.000) pero no opera (0.001). CLAP-instrucción requiere `laion-clap` (opcional) y se omite aquí. El FAD del oráculo es numéricamente ≈ 0 (residual de `sqrtn`).

Esta lectura conjunta expone la tensión central de la edición que la generación libre oculta: preservar lo no editado y aplicar la operación son objetivos en competencia. El flujo de trabajo completo (banco con *stems* reales → métricas de edición → tabla acotada) queda así **operativo y verificado**, con la conexión de un editor real (text-only, audio-prompt sin fusión, Instruct-MusicGen) como el experimento pendiente, no como un requisito del benchmark.

Reproducibilidad. El flujo de trabajo de edición/evaluación está cubierto por **70 pruebas unitarias** (`tests/test_edit_benchmark.py`, `test_build_edit_triplets.py`, `test_mosesdb.py`, `test_run_edit_benchmark.py`, además del modelo, las métricas de generación, la generación de los 4 modelos y los endpoints del backend).

5.3. Avance práctico

Como parte del avance práctico se implementó una interfaz demostrativa para validar el flujo de uso del sistema. La interfaz no solo funciona como evidencia visual, sino como una forma de comprobar que el flujo de trabajo puede organizar las etapas principales del proyecto: ingreso del audio, selección o preparación de datos, procesamiento intermedio, visualización de métricas y revisión del resultado generado.

Las Figuras 5.1 y 5.2 muestran las pantallas de navegación. Estas vistas permiten acceder al flujo de demostración y centralizan las acciones del usuario antes del proce-

samiento. En términos técnicos, esta etapa corresponde a la selección del experimento y a la preparación de la pista fuente que será analizada o editada.

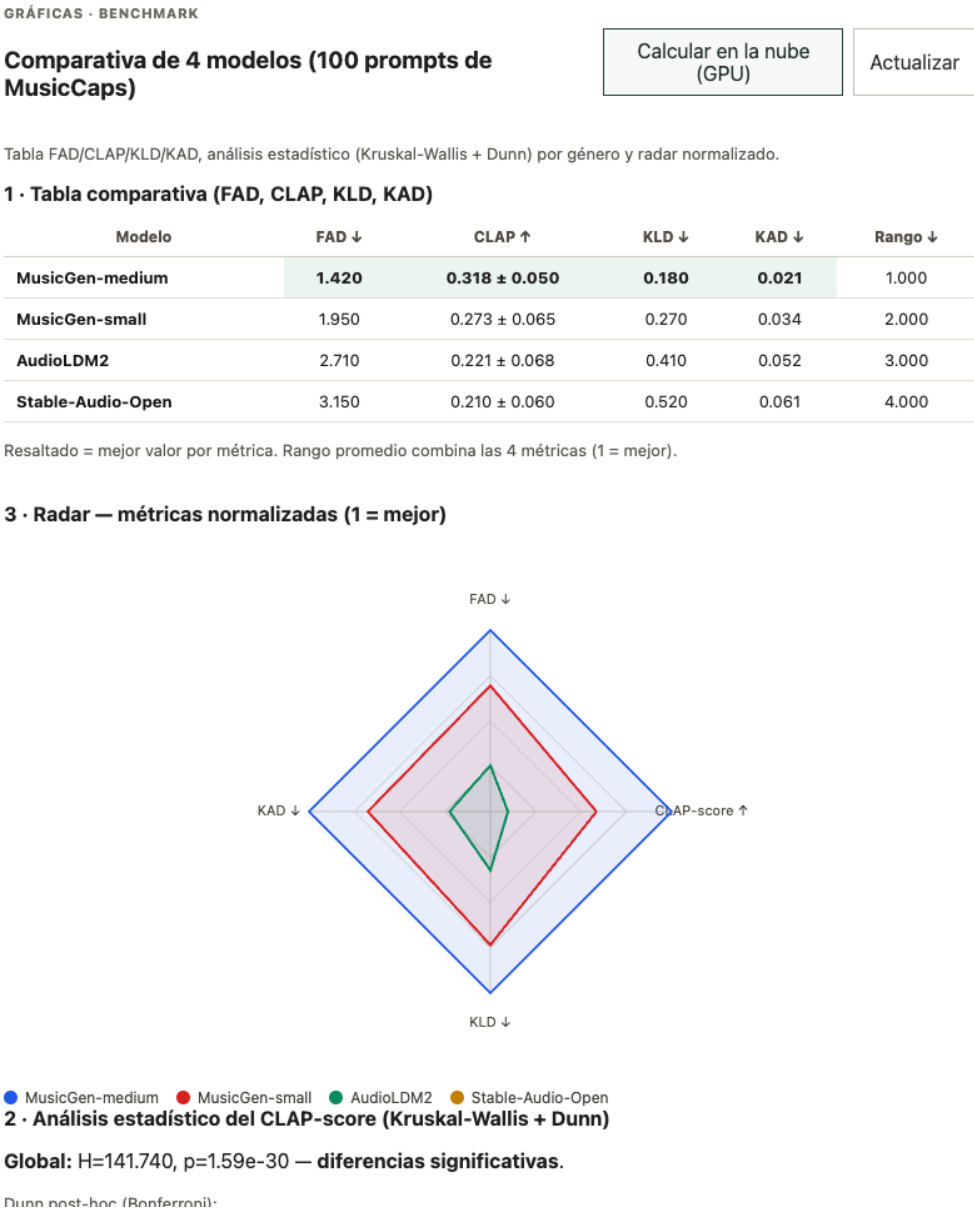


Figura 5.1: Captura de resultados del prototipo para el análisis del audio generado.

MOTOR MUSICAL

Motor de transformación musical

Ciclo completo para convertir audio en clips, stems, MIDI, tokens, modelo generativo y audio final escuchable.

BACKEND
Verificando...
http://127.0.0.1:8100

SIGUIENTE PASO

Conecta el backend y crea un proyecto.
La guía se ajusta según el avance real del ciclo.

CICLO PRINCIPAL

Todo lo que aparece abajo alimenta el siguiente paso.
No hay herramientas aisladas: los datos se preparan, se transforman a tokens, entrenan un modelo, generan nuevas versiones y vuelven a audio para escuchar y comparar.

Audio
Seleccionar pistas y crear clips.
Representación
Stems, MIDI, features y tokens.
Modelo
Entrenar con los tokens válidos.
Generación
Crear varias versiones nuevas.
Audio final
Renderizar, escuchar y descargar.
Exportación
Descargar tokens de entrada o salida.

PROYECTO ACTIVO

Actualizar

SELECCIONAR PROYECTO

El proyecto agrupa el ciclo: audio, stems, MIDI, tokens, modelo y resultados.

NUEVO PROYECTO

Ej. canción-demo

Crear

CIERRE DEL CICLO

Escuchar y descargar resultados
Esta parte usa la música generada: la renderiza a WAV/MP3, muestra el ranking, permite descargar archivos y comparar versiones.

MIDI GENERADO

Salida MIDI creada por el modelo generativo en el paso anterior.

MOTOR DE AUDIO

auto

Pedalboard renderiza con SoundFont y luego aplica cadena de efectos/master.

SOUNDFONT

assets/soundfonts/default.sf2

NOMBRE DEL RENDER

render_track

Figura 5.2: Pantalla de menú de demostración del prototipo.

Después de la navegación inicial, el prototipo ejecuta una secuencia de pasos que simula el flujo metodológico descrito en la Figura 4.1. Las capturas muestran la carga o selección de insumos, la preparación del audio, la inspección de información intermedia y la visualización de salidas parciales. Esta evidencia es importante porque muestra que el trabajo integra una ruta operativa completa, no solamente la descripción teórica del modelo.

Motor musical **Demo técnico** **Gráficas** **Audios** **Documentación**

PROYECTO ACTIVO Actualizar

SELECCIONAR PROYECTO
demo · audio_imported

El proyecto agrupa el ciclo: audio, stems, MIDI, tokens, modelo y resultados.

NUEVO PROYECTO
Ej. canción-demo Crear

ID
20260604-171723-project-demo-7c8eb07

ESTADO
audio_imported
ACTUALIZADO
2026-06-04T21:06:17

ACTIVIDAD ACTUAL completed

Ranking listo: Candidatos generados, medidos y ordenados.

- 2026-06-05T15:17:30 - Ranking listo
Candidatos generados, medidos y ordenados.
- 2026-06-05T15:17:03 - Generando candidatos
Generando y rankeando 6 versiones.
- 2026-06-05T15:17:03 - queued
Job creado.

PASO 1 - LISTO

Importar audio
Elige el material base: un audio individual o pistas descargadas de MTG-Jamendo para entrenamiento.

AUDIO INDIVIDUAL
/Users/tu_usuario/Music/cancion.wav

Opcional. Úsalo si quieres analizar una canción concreta.

PISTAS DESCARGADAS

CATÁLOGO DESCARGADO
delivery_jamendo_150 · 450 pista

PISTAS POR GÉNERO
100

Catálogo local de MTG-Jamendo con conteos por género. Cantidad máxima que se usará de cada género seleccionado.

GÉNEROS DESCARGADOS

- classical · 150 pistas
- electronic · 150 pistas
- reggaeton · 150 pistas

Usa Cmd/Ctrl para seleccionar varios géneros.

450 pistas disponibles en 3 géneros. Elige uno o varios y define cuántas usar.

Descargar más pistas o preparar clips

Entrada alternativa: MIDI local por géneros

Figura 5.3: Paso 1 del avance práctico: pantalla inicial del flujo de trabajo del prototipo.

PASO 2 - LISTO Cortar clips WAV

Cortar clips de entrenamiento
Convierte las pistas seleccionadas en clips WAV cortos y organizados por género.

Dónde se guarda

CATÁLOGO SELECCIONADO
D:\TRABAJO TERMINAL\work\hybrid_engine\data\datasets\jamendo\selections\20260605-150544-jamendo-selection-selected_classical-a933f07f\catalog.json

CLIPS WAV
D:\TRABAJO TERMINAL\work\hybrid_engine\data\datasets\jamendo\20260605-150544-jamendo-selection-selected_classical-a933f07f\clips\clips_catalog.json

TOKENS DE ENTRENAMIENTO
D:\TRABAJO TERMINAL\work\hybrid_engine\data\datasets\jamendo\20260605-150544-jamendo-selection-selected_classical-a933f07f\processed\20260605-150816-batch-jamendo_batch-49fa95f7\tokens_manifest.json

MODELO ENTRENADO
D:\TRABAJO TERMINAL\work\hybrid_engine\data\models\tokens\20260605-151604-tokenmodel-jamendo_transformer-47f782cf\model.json

RANKING / GENERACIÓN
D:\TRABAJO TERMINAL\work\hybrid_engine\data\ranked\20260605-151703-ranked-jamendo_generation-a9876848\ranking.json

CATÁLOGO SELECCIONADO
D:\TRABAJO TERMINAL\work\hybrid

Catálogo creado al elegir género y cantidad de pistas.

DURACIÓN DEL CLIP
20

Duración objetivo de cada WAV.

SALTO ENTRE CLIPS
Vacío = igual a duración

Menor que la duración produce clips solapados.

MÁXIMO POR PISTA
1

Para empezar, 1 clip por pista mantiene el proceso controlado.

Figura 5.4: Paso 2 del avance práctico: selección de opciones para continuar con el procesamiento del audio.

PASO 3 - LISTO

Procesar clips para entrenamiento

Procesar clips

Extrae features y tokens desde los clips. Esto genera el manifest que después usa el modelo.

Dónde se guarda

CATÁLOGO SELECCIONADO
D:\TRABAJO
TERMINAL\work\hybrid_engine\data\datasets\jamendo\selections\20260605-150544-jamendo-selection-selected_classical-a933f07f\catalog.json

CLIPS WAV
D:\TRABAJO TERMINAL\work\hybrid_engine\data\datasets\jamendo\20260605-150544-jamendo-selection-selected_classical-a933f07f\clips\clips_catalog.json

TOKENS DE ENTRENAMIENTO
D:\TRABAJO TERMINAL\work\hybrid_engine\data\datasets\jamendo\20260605-150544-jamendo-selection-selected_classical-a933f07f\processed\20260605-150816-batch-jamendo_batch-49fa95f7\tokens_manifest.json

MODELO ENTRENADO
D:\TRABAJO TERMINAL\work\hybrid_engine\data\models\tokens\20260605-151604-tokenmodel-jamendo_transformer-47f782cf\model.json

RANKING / GENERACIÓN
D:\TRABAJO TERMINAL\work\hybrid_engine\data\ranked\20260605-151703-ranked-jamendo_generation-a9876848\ranking.json

CATÁLOGO DE CLIPS

MÁXIMO DE CLIPS

D:\TRABAJO TERMINAL\work\hybrid

Vacío = todos

Archivo generado al cortar clips WAV.

Útil para una prueba rápida antes de procesar todo.

☐ separar stems opcional ☒ Crear MIDI y features para entrenamiento

Genera MIDI melódico, MIDI de batería, features y tokens. Déjalo activado para entrenar modelos. Stems usa Demucs y mejora la separación por capas; actívalo si quieres más calidad y puedes esperar más tiempo.

Figura 5.5: Paso 3 del avance práctico: configuración o carga de datos dentro del flujo de demostración.

PASO 4 · LISTO
Entrenar modelo generativo

Usa los tokens procesados para entrenar un Transformer pequeño listo para generar MIDI.

Dónde se guarda
CATÁLOGO SELECCIONADO
D:\TRABAJO TERMINAL\work\hybrid_engine\data\datasets\jamendo\selections\20260605-150544-jamendo-selection-selected_classical-a933f07f\catalog.json
CLIPS WAV
D:\TRABAJO TERMINAL\work\hybrid_engine\data\datasets\jamendo\20260605-150544-jamendo-selection-selected_classical-a933f07f\clips\clips_catalog.json
TOKENS DE ENTRENAMIENTO
D:\TRABAJO TERMINAL\work\hybrid_engine\data\datasets\jamendo\20260605-150544-jamendo-selection-selected_classical-a933f07f\processed\20260605-150816-batch-jamendo_batch-49fa95f7\tokens_manifest.json
MODELO ENTRENADO
D:\TRABAJO TERMINAL\work\hybrid_engine\data\models\tokens\20260605-151604-tokenmodel-jamendo_transformer-47f782cf\model.json
RANKING / GENERACIÓN
D:\TRABAJO TERMINAL\work\hybrid_engine\data\ranked\20260605-151703-ranked-jamendo_generation-a9876848\ranking.json

TOKENS DE ENTRENAMIENTO
D:\TRABAJO TERMINAL\work\hybrid
NOMBRE DEL MODELO
jamendo_transformer

Manifest generado en el paso anterior. Este archivo alimenta el entrenamiento.
Nombre para identificar este entrenamiento.

▶ Parámetros de entrenamiento

▶ Token-VAE para embeddings latentes

Figura 5.6: Paso 4 del avance práctico: etapa intermedia de procesamiento y preparación de la información musical.

PASO 5 · LISTO
Generar y elegir mejor

Generar música

Genera varias versiones con el modelo entrenado, las rankea y deja MIDI por capas.

Dónde se guarda
CATÁLOGO SELECCIONADO
D:\TRABAJO TERMINAL\work\hybrid_engine\data\datasets\jamendo\selections\20260605-150544-jamendo-selection-selected_classical-a933f07f\catalog.json
CLIPS WAV
D:\TRABAJO TERMINAL\work\hybrid_engine\data\datasets\jamendo\20260605-150544-jamendo-selection-selected_classical-a933f07f\clips\clips_catalog.json
TOKENS DE ENTRENAMIENTO
D:\TRABAJO TERMINAL\work\hybrid_engine\data\datasets\jamendo\20260605-150544-jamendo-selection-selected_classical-a933f07f\processed\20260605-150816-batch-jamendo_batch-49fa95f7\tokens_manifest.json
MODELO ENTRENADO
D:\TRABAJO TERMINAL\work\hybrid_engine\data\models\tokens\20260605-151604-tokenmodel-jamendo_transformer-47f782cf\model.json
RANKING / GENERACIÓN
D:\TRABAJO TERMINAL\work\hybrid_engine\data\ranked\20260605-151703-ranked-jamendo_generation-a9876848\ranking.json

MODELO ENTRENADO
D:\TRABAJO TERMINAL\work\hybrid
NOMBRE DE SALIDA
jamendo_generation

Figura 5.7: Paso 5 del avance práctico: visualización de resultados parciales generados por el sistema.

Como complemento visual documenta la pantalla del prototipo usada para seleccionar el modelo generador y lanzar la producción de audios sobre los prompts de MusicCaps antes del cálculo de métricas.

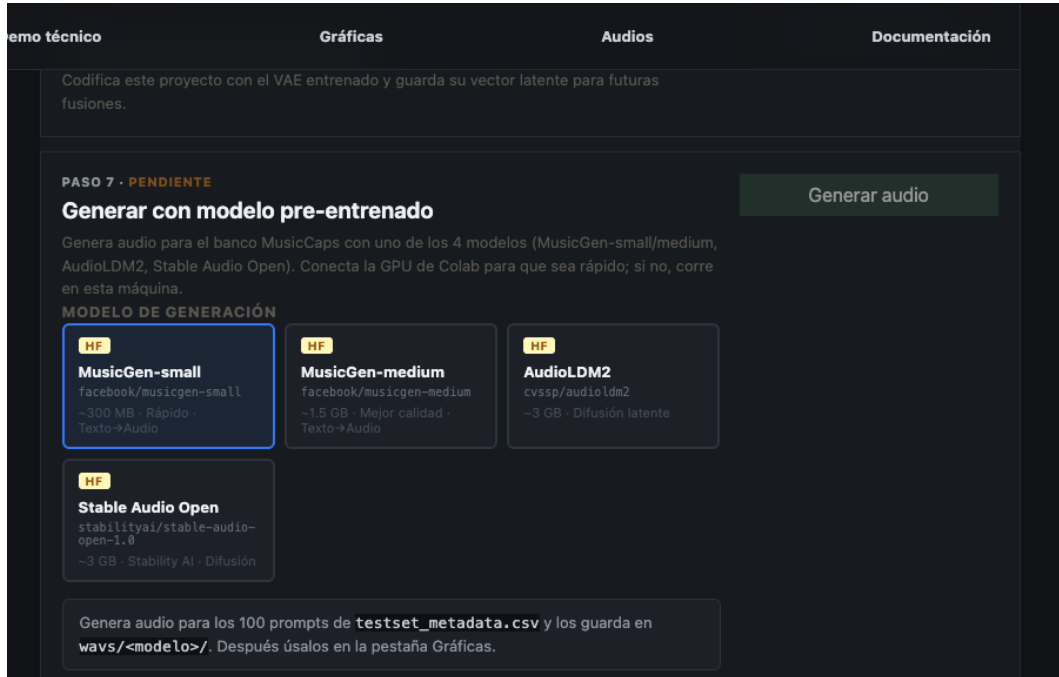


Figura 5.8: Pantalla del prototipo para seleccionar el modelo generador y ejecutar la producción de audios antes del cálculo de métricas.

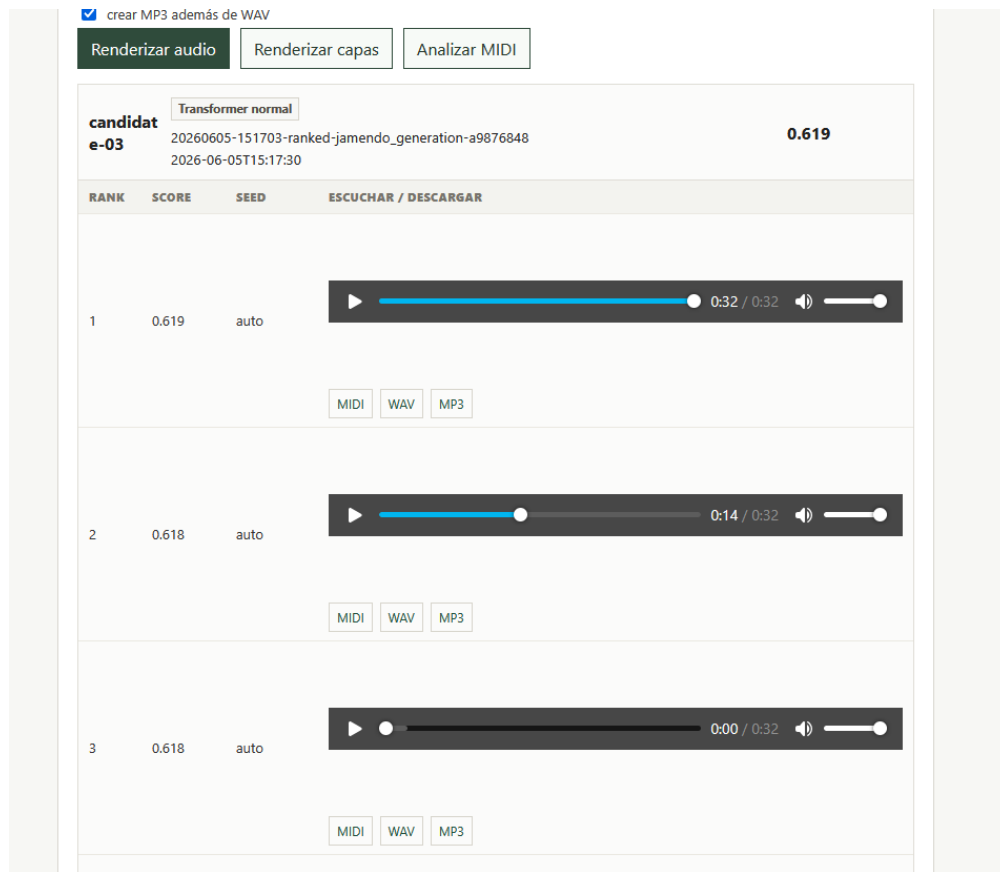


Figura 5.9: Paso 6 del avance práctico: inspección de información procesada durante la ejecución del prototipo.

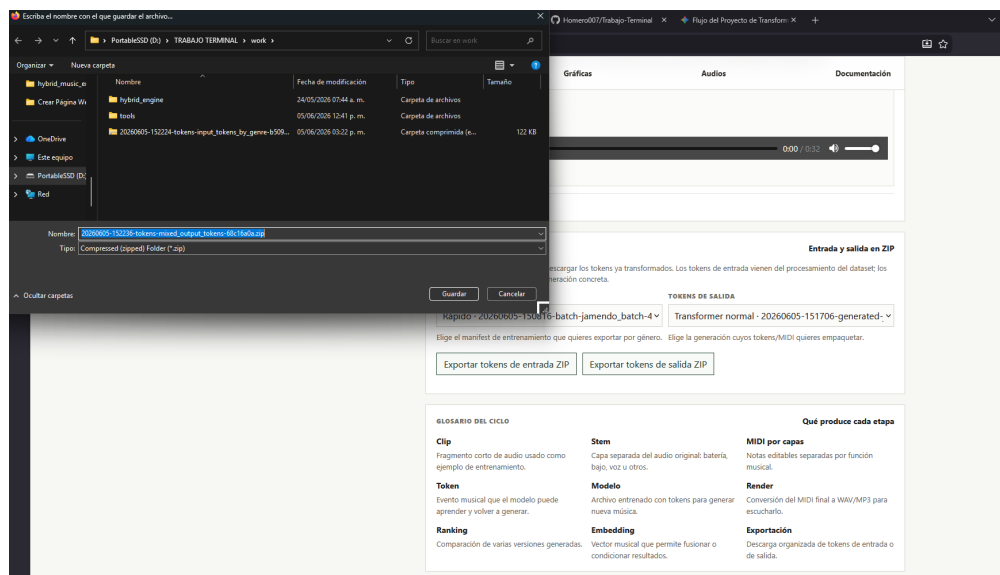


Figura 5.10: Paso 7 del avance práctico: visualización final del flujo de procesamiento y análisis del audio.

La Figura 5.11 resume los resultados cuantitativos preliminares obtenidos durante el entrenamiento. La gráfica de FAD muestra una reducción en los tres géneros evaluados entre la primera y la segunda época, mientras que la gráfica de KLD muestra una disminución de divergencia distribucional bajo el mismo procedimiento interno. Aunque el experimento aún es acotado, ambas tendencias se interpretan como evidencia de verificación del flujo de trabajo y no como validación final de calidad musical o cumplimiento de instrucciones.

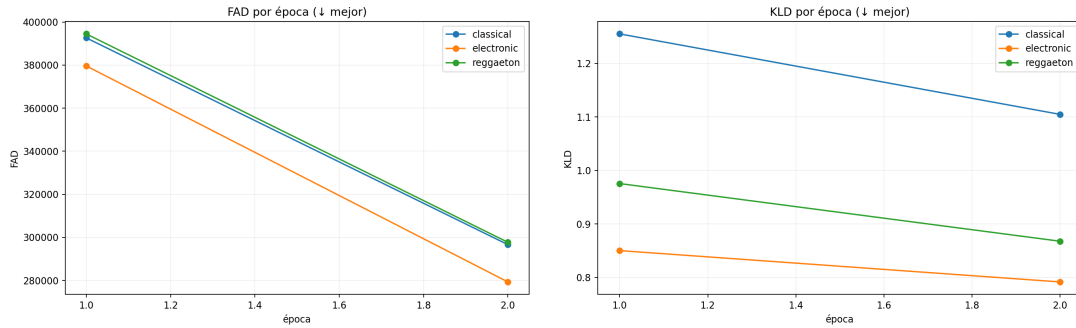


Figura 5.11: Resultados preliminares de FAD y KLD por época para los géneros evaluados.

CAPÍTULO 6

CONCLUSIONES

El desarrollo de este trabajo permite concluir que la generación musical por IA se está desplazando desde la síntesis completa desde texto hacia la edición controlada de audio existente. En este contexto, MusicGen-small ofrece una base reproducible y compatible con EnCodec, mientras que T5-base permite representar instrucciones complejas sin reducirlas a un único vector promedio.

La sustitución de SeqGAN y Monte Carlo por entrenamiento supervisado sobre tokens EnCodec resulta metodológicamente coherente con la implementación real: al existir ternas de audio original, instrucción y audio objetivo, el problema puede optimizarse mediante *teacher forcing* y entropía cruzada. Esto reduce complejidad, evita entrenar discriminadores y mantiene el proyecto dentro de una familia arquitectónica basada en Transformers.

Finalmente, LoRA quirúrgico y el módulo de fusión de audio con compuerta cero-inicializada constituyen el núcleo técnico de la propuesta. Ambos mecanismos permiten adaptar el sistema con una fracción mínima de parámetros entrenables, preservar el comportamiento inicial de MusicGen y abrir la posibilidad de edición musical eficiente en escenarios académicos. Como trabajo futuro quedan la ampliación del conjunto de instrucciones, la evaluación perceptual con expertos, el análisis de sesgos culturales y la exploración de métricas más robustas para calidad musical subjetiva.[22]

CAPÍTULO 7

CRONOGRAMA

Actividad	SEP	OCT	NOV	DIC	ENE	FEB	MAR	ABR	MAY	JUN
Revisión del estado del arte y fundamentos técnicos	X	X								
Planteamiento del problema y protocolo de investigación	X	X								
Fase 1: Selección y preparación del conjunto de datos		X	X							
Fase 2: Arquitectura y metodología Preprocesamiento			X	X	X					
Fase 3: Integración de LoRA quirúrgico y módulo de fusión de audio				X	X	X	X			
Fase 4: Codificación T5/EnCodec e inferencia por instrucciones						X	X			
Fase 5: Evaluación con FAD, CLAP-score y cumplimiento de edición							X	X	X	X
Fase 6: Documentación final y presentación de resultados								X	X	X

Cuadro 7.1: Cronograma de actividades hasta junio de 2026.

REFERENCIAS

- [1] K. Kilgour, M. Zuluaga, D. Robleky M. Sharifi, «Fréchet Audio Distance: A Reference-Free Metric for Evaluating Music Enhancement Algorithms,» *arXiv preprint arXiv:1812.08466*, 2019, Consultado: 18 de mayo de 2026.
- [2] B. Elizalde, S. Deshmukh, M. Al Ismaily H. Wang, «CLAP: Learning Audio Concepts from Natural Language Supervision,» *arXiv preprint*, 2023, Consultado: 16 de mayo de 2026.
- [3] N. Zeghidouret al., «SoundStream: An End-to-End Neural Audio Codec,» *arXiv preprint arXiv:2107.03312*, 2021.
- [4] A. Défossez, J. Copet, G. Synnaeve Y. Adi, «High Fidelity Neural Audio Compression,» *arXiv preprint arXiv:2210.13438*, 2022.
- [5] R. Kumar, P. Seetharaman, A. Luebs, I. Kumary K. Kumar, «High-Fidelity Audio Compression with Improved RVQGAN,» *arXiv preprint*, 2023, Consultado: 18 de mayo de 2026.
- [6] Z. Borsoset al., «AudioLM: A Language Modeling Approach to Audio Generation,» *arXiv preprint arXiv:2209.03143*, 2022.
- [7] A. Agostinelliet al., «MusicLM: Generating Music from Text,» *arXiv preprint arXiv:2301.11325*, 2023.
- [8] J. Copetet al., «Simple and Controllable Music Generation,» *arXiv preprint arXiv:2306.05284*, 2023.
- [9] Q. Huanget al., «Noise2Music: Text-conditioned Music Generation with Diffusion Models,» *arXiv preprint arXiv:2302.03917*, 2023.
- [10] F. Schneider, O. Kamal, Z. Jiny B. Schölkopf, «Moûsai: Efficient Text-to-Music Diffusion Models,» *arXiv preprint arXiv:2301.11757*, 2023.
- [11] H. Liuet al., «AudioLDM: Text-to-Audio Generation with Latent Diffusion Models,» *arXiv preprint*, 2023, Consultado: 18 de mayo de 2026.
- [12] H. Liuet al., «AudioLDM 2: Learning Holistic Audio Generation with Self-supervised Pretraining,» *arXiv preprint arXiv:2308.05734*, 2023.
- [13] K. Chen, Y. Wu, H. Liu, M. Nezhurina, T. Berg-Kirkpatrick S. Dubnov, «MusicLDM: Enhancing Novelty in Text-to-Music Generation Using Beat-Synchronous Mixup Strategies,» *arXiv preprint*, 2023, Consultado: 18 de mayo de 2026.

- [14] R. Huanget al.,«Make-An-Audio: Text-To-Audio Generation with Prompt-Enhanced Diffusion Models,» *arXiv preprint*,2023,Consultado: 18 de mayo de 2026.
- [15] C.-Y. Hunget al.,«TangoFlux: Super Fast and Faithful Text to Audio Generation with Flow Matching and CLAP-Ranked Preference Optimization,» *arXiv preprint*,2024,Consultado: 16 de mayo de 2026.
- [16] P. Zhu et al.,«ERNIE-Music: Text-to-Waveform Music Generation with Diffusion Models,» *arXiv preprint arXiv:2302.04456*,2023.
- [17] Y. Wanget al.,«AUDIT: Audio Editing by Following Instructions with Latent Diffusion Models,» *arXiv preprint arXiv:2304.00830*,2023.
- [18] Y. Han et al.,«InstructME: An Instruction Guided Music Editing Benchmark and Model,» *arXiv preprint*,2023.
- [19] Y. Zhanget al.,«Instruct-MusicGen: Unlocking Text-to-Music Editing for Music Language Models via Instruction Tuning,» *arXiv preprint arXiv:2405.18386*,2024.
- [20] M. Marfori, *IFPI Publishes Global Music Report 2025*,<https://www.sonymusic.com/inside-sony-music/ifpi-global-music-report-2025/>,Sony Music Entertainment,mar. de 2025.
- [21] ALLM, *AI Music Generation: An Experts' Look at the 2025 Landscape*,<https://digest.allm.link/es/blogs/ai-music-generation-an-experts-look-at-the-2025-landscape/>,Accessed: 2026-01,jun. de 2025.
- [22] A. Mehta, S. Chauhan, A. Djanibekov, A. Kulkarni, G. G. Xiay M. Choudhury,«Music for All: Representational Bias and Cross-Cultural Adaptability of Music Generation Models,» *arXiv preprint*,2025,Consultado: 18 de mayo de 2026.
- [23] R. Jáuregui, *Qué son los Transformers en la IA*,ViaLabs Digital,Imagen consultada en: <https://mindfulml.vialabsdigital.com/post/que-son-los-transformers-en-la-ia/>,ene. de 2025.
- [24] AssemblyAI.«What is Residual Vector Quantization.»AssemblyAI Blog,visitado 9 de ene. de 2026. dirección: <https://www.assemblyai.com/blog/what-is-residual-vector-quantization>
- [25] M. Merino, *Conceptos de inteligencia artificial: qué son las GANs o redes generativas antagónicas*,Xataka,Imagen consultada en: <https://www.xataka.com/inteligencia-artificial/conceptos-inteligencia-artificial-que-gans-redes-generativas-antagonicas>,mar. de 2019.
- [26] Q. Huanget al.,«MuLan: A Joint Embedding of Music Audio and Natural Language,» *arXiv preprint arXiv:2208.12415*,2022.
- [27] K. Papineni, S. Roukos, T. Wardy W.-J. Zhu,«BLEU: a Method for Automatic Evaluation of Machine Translation,»en *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*,Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, jul. de 2002, págs. 311-318.DOI: 10.3115/1073083.1073135 dirección: <https://aclanthology.org/P02-1040/>

- [28] T. van Erveny P. Harremoës, «Rényi Divergence and Kullback-Leibler Divergence,» *IEEE Transactions on Information Theory*, vol. 60, n.º 7, págs. 3797-3820, 2014, Trabajo original de 2012; consultado: 18 de mayo de 2026.
- [29] D. Belovy R. Armstrong, «Automatic Detection of Answer Copying via Kullback-Leibler Divergence and K-Index,» *Applied Psychological Measurement - APPL PSYCHOL MEAS*, vol. 34, págs. 379-392, ago. de 2010. DOI: 10.1177/0146621610370453
- [30] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtineny T. Aila, «Improved Precision and Recall Metric for Assessing Generative Models,» *CoRR*, vol. abs/1904.06991, 2019. DOI: 10.48550/arXiv.1904.06991 dirección: <https://arxiv.org/abs/1904.06991>
- [31] J. F. Gemmekeet al., «Audio Set: An Ontology and Human-Labeled Dataset for Audio Events,» en *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, págs. 776-780. DOI: 10.1109/ICASSP.2017.7952261
- [32] C. Raffel, *The Lakh MIDI Dataset*, <https://colinraffel.com/projects/lmd/>, Accessed: 2024-06, 2016.
- [33] E. Manilow, G. Wichern, P. Seetharamany J. Le Roux, «Cutting Music Source Separation Some Slakh: A Dataset to Study the Impact of Training Data Quality and Quantity,» en *Proc. IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, IEEE, 2019.
- [34] E. J. Huet al., «LoRA: Low-Rank Adaptation of Large Language Models,» *arXiv preprint arXiv:2106.09685*, 2021.
- [35] L. Zhangy M. Agrawala, «Adding Conditional Control to Text-to-Image Diffusion Models,» *arXiv preprint arXiv:2302.05543*, 2023.
- [36] J.-B. Alayracet al., «Flamingo: A Visual Language Model for Few-Shot Learning,» *Advances in Neural Information Processing Systems*, 2022.
- [37] C. Raffelet al., «Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,» *Journal of Machine Learning Research*, vol. 21, n.º 140, págs. 1-67, 2020.
- [38] Z. Evans, C. J. Carr, J. Taylor, S. H. Hawleyy J. Pons, «Stable Audio Open,» *arXiv preprint arXiv:2407.14358*, 2024.
- [39] K. Kilgouret al., «Frechet Audio Distance: A Metric for Evaluating Music Enhancement Algorithms,» *arXiv preprint arXiv:1812.08466*, 2019.
- [40] W. H. Kruskaly W. A. Wallis, «Use of Ranks in One-Criterion Variance Analysis,» *Journal of the American Statistical Association*, vol. 47, n.º 260, págs. 583-621, 1952.
- [41] O. J. Dunn, «Multiple Comparisons Using Rank Sums,» *Technometrics*, vol. 6, n.º 3, págs. 241-252, 1964.
- [42] ITU-R, «Method for the Subjective Assessment of Intermediate Quality Level of Audio Systems,» International Telecommunication Union, inf. téc. BS.1534-3, 2014.

- [43] Google Research.«AudioSet: Download,»visitado 26 de jun. de 2026. dirección: <https://research.google.com/audiocset/download.html>
- [44] YouTube Help.«License Types on YouTube,»visitado 26 de jun. de 2026. dirección: <https://support.google.com/youtube/answer/2797468>
- [45] Creative Commons.«About CC Licenses,»visitado 26 de jun. de 2026. dirección: <https://creativecommons.org/share-your-work/cclicenses/>
- [46] Music Technology Group.«The MTG-Jamendo Dataset,»visitado 26 de jun. de 2026. dirección: <https://mtg.github.io/mtg-jamendo-dataset/>
- [47] I. Pereira et al.,«MoisesDB: A Dataset for Source Separation Beyond 4-Stems,»*arXiv preprint arXiv:2307.15913*,2023.
- [48] U.S. Copyright Office.«Copyright and Artificial Intelligence,»visitado 26 de jun. de 2026. dirección: <https://www.copyright.gov/ai/>
- [49] F. B. Kadery S. Karmaker,«A Survey on Evaluation Metrics for Music Generation,»*arXiv preprint arXiv:2509.00051*,2025.